

NewtonX ADK開発環境構築ガイド (v2.1)

概要

このガイドでは、NewtonX ADKを使用したアプリケーション開発において、Python仮想環境（venv）を活用する方法を説明します。venvを使用することで、NewtonX ADK専用のクリーンな開発環境を構築し、他のプロジェクトとの依存関係の競合を防ぐことができます。

本ガイドはWindowsを主対象としつつ、可能な箇所にmacOS向けのコマンド・ショートカットも併記しています。

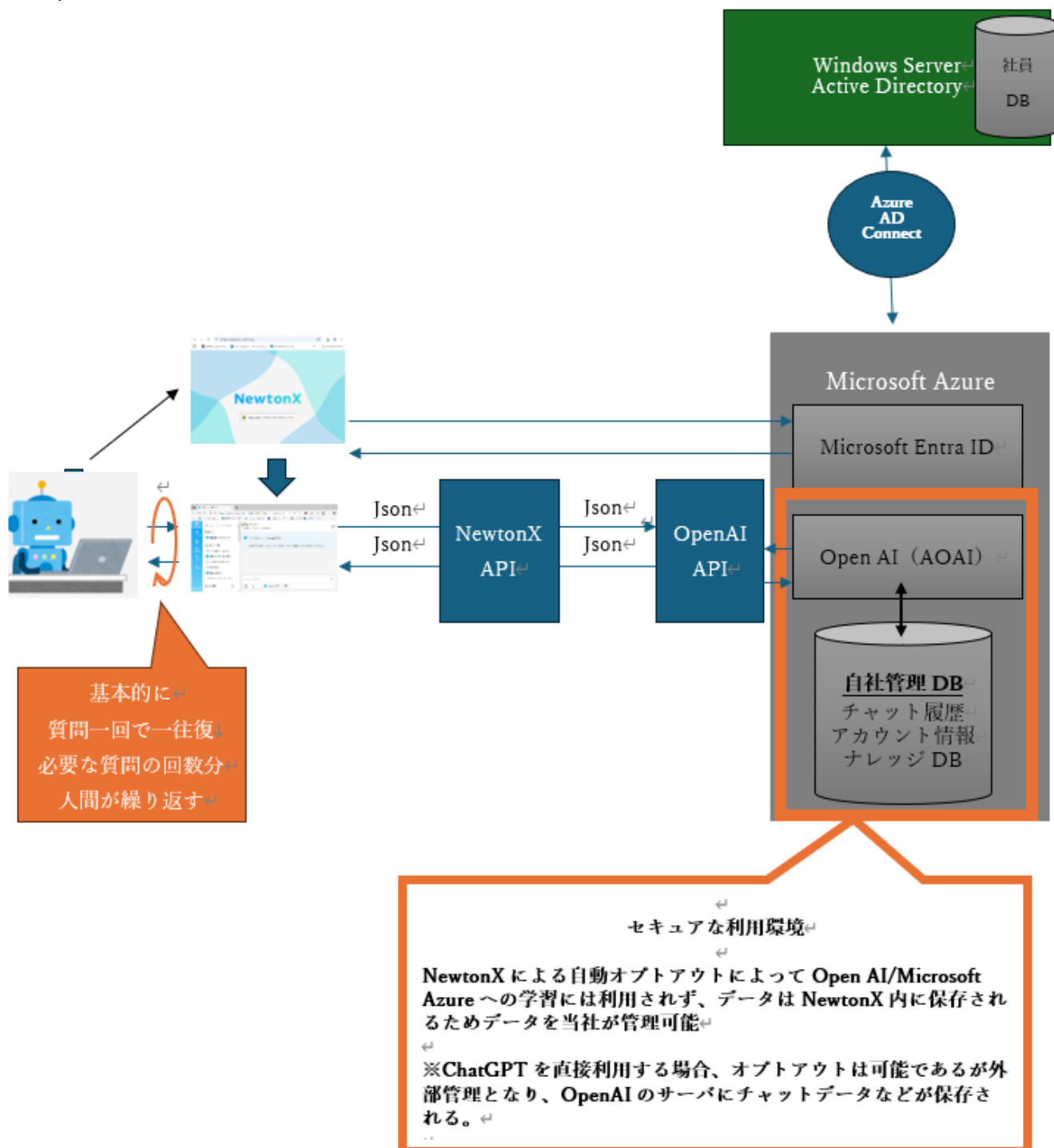
前提条件

- Windows環境ではPython 3.13以降（推奨: 3.14）をインストールする必要があります。本ガイドでは**Python Install Manager（py.exe）**を使用したインストール方法を説明します。
 - macOSやLinuxではPythonがデフォルトでインストールされている場合が多いですが、バージョンが古い場合や複数バージョンを管理したい場合は新たにインストールしてください。
 - NewtonX ADKの利用申請が完了していること（3.4節参照）。
 -  は環境や実行に最小限必要な実施項目です。
 - コマンド入力はWindowsのコマンドプロンプトで操作してください。（PowerShellではそのままでは実行できないコマンドがあります）
-

1. NewtonX ADK (Agent Development Kit) について

1.1 NewtonXとは

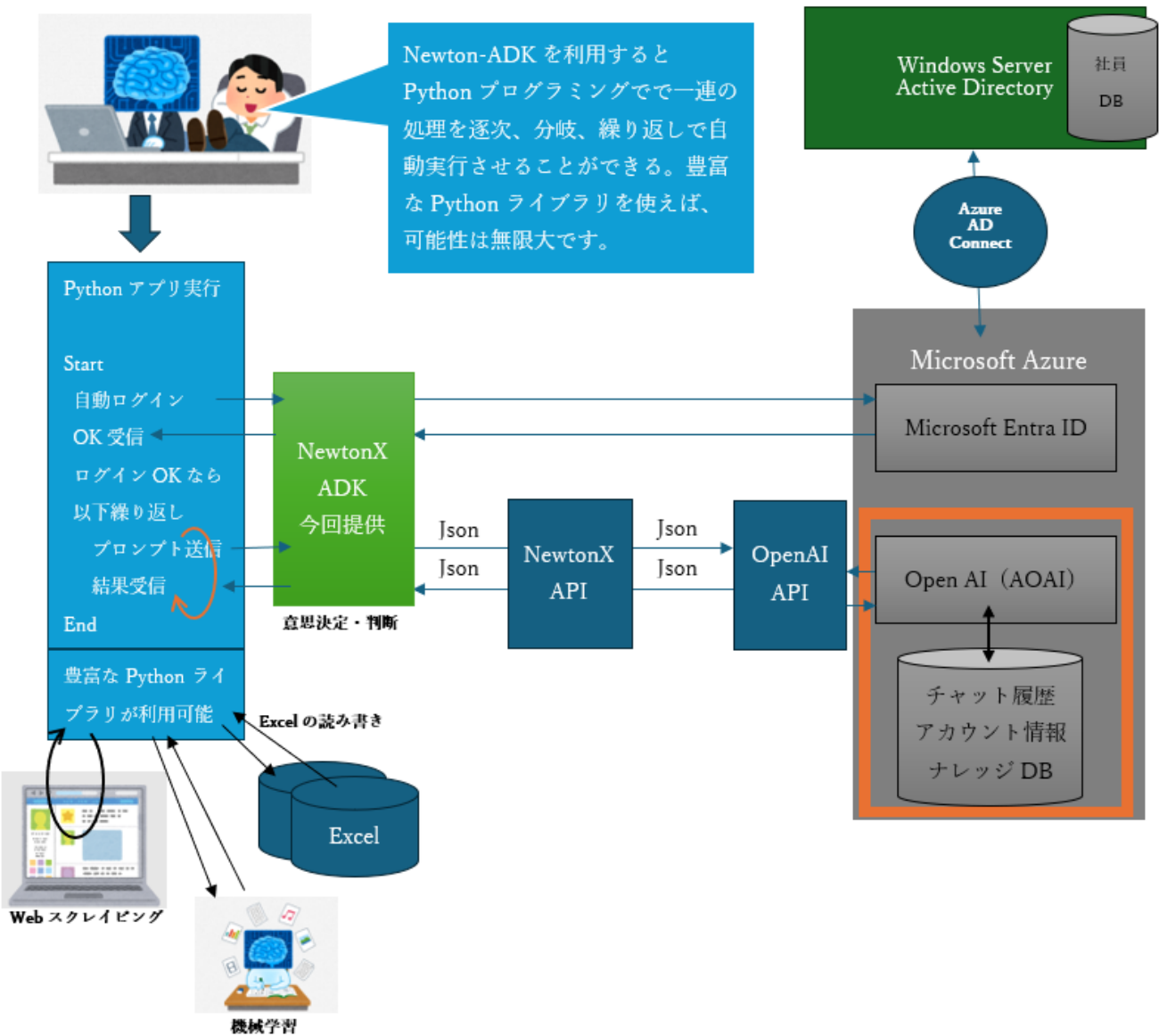
Azure OpenAI Service上で自社で開発された、安全性の高いセキュリティを実現した法人向け生成AIプロダクト



1.2 NewtonX ADKとは

ADKとはAgent Development Kitの略で、NewtonX（ニュートンエックス）のAIアシスタントと簡単にやり取りできるPythonライブラリです。このライブラリを使うと、PythonプログラムからNewtonXのAIアシスタントとチャットしたり、ファイルを送ったりするプログラムを作成することができます。

日々の業務プロセスの一部または大部分を安全に代行してくれるアプリケーションを作成できる環境を提供します。



1.3 環境構築の全体ステップ

本ガイドでは、NewtonX ADKを使用したアプリケーション開発環境を構築するため、以下の5つのステップを順に実施します。文中の📖マークは最低限実施が必要な作業項目です。

ステップ	作業内容	説明	準備事項
2章	Pythonのインストール📖	Python Install ManagerとPythonのインストールおよびレガシーなPython Launcherの削除	特になし
3章	NewtonX ADK開発環境の構築📖	プロジェクトフォルダの作成とNewtonX ADKのインストール	NewtonX ADK利用申請後にNewtonX ADK配布サイトよりNewtonX ADKのzipファイルとVSCodeのconfig用jsonのzipファイルをバッチファイルと同じフォルダに配置しておくこと 例) newtonx_adk-0.10.5.zipとjson_ver1.1.zip
4章	VSCodeのインストールとPython開発環境設定📖	VSCodeのインストールおよび拡張機能のインストール、Python開発環境の設定	VSCodeのインストール、拡張機能のインストール、仮想環境の有効化、プロジェクトの読み込みを行ってVSCodeを起動します。
5章	認証プログラムの実行📖	NewtonX ADK用認証プログラムの実行	WEBブラウザでNewtonXを起動して、アクセストークンの発行が必要です。HOST名の入力には注意してください
6章	サンプルアプリの実行📖	サンプルアプリ (cli_example など) の実行	adk_examples配下の3つのアプリの依存関係をインストールし、サンプルを実行します。

注意事項📖

- コマンドプロンプト操作が可能な方は、📖マークの項目を必ず実施してください。
- 各ステップは順番通りに実施することを推奨します
- 仮想環境を使用することで、クリーンな開発環境を維持できます
- 問題が発生した場合は、該当する章の「注意事項」セクションを参照してください
- コマンド入力のところでは、以下の手順で**コマンドプロンプト**、または、**ターミナル**を起動してください。
 1. **Windowsの場合:**📖
 - Windowsの下の検索窓で、cmdと入力し起動します。
 - 説明の際の画面はユーザをserakuとして操作説明をしていますので自身のユーザ名と読み替えてください。

2. Pythonのインストール📖

2.1 Python Install Manager と Python のインストール📖

2.1.1 Python Install Managerとは（概要）

- **説明:** Python Install Manager for Windows (**py.exe**ランチャー) は、Windows向けの公式ツールです。複数のPythonバージョンをPC上に共存させ、簡単に切り替えて利用することを可能にします。
- **主な特徴:**
 - **複数バージョンの管理:** **py install <version>** コマンドで簡単に新しいバージョンをインストールできます。
 - **シンプルな実行:** **py** コマンドでデフォルトのPythonを起動できます。
 - **バージョン指定実行:** **py -3.14** のように、特定のバージョンを指定してスクリプトを実行できます。
 - **Microsoft Store対応:** Microsoft Store経由でのPython管理とも連携します。
- **従来の.exeインストーラとの関係:** 既に従来の.exeインストーラでPython 3.13や3.14などをインストールしている場合でも、**Python Install Manager**を後から追加インストールして共存させることが可能です。ただし、それぞれが別々のPython環境を管理するため、「どのコマンドがどの環境を使っているか」を意識して使い分ける必要があります。
 - **従来インストーラ由来:** **python** コマンドで直接起動されることが多い。
 - **Install Manager由来:** **py** コマンドでバージョンを切り替えて利用 (**py -3.14** など)。

どちらが使われているか分からなくなった場合は、**where python** や **py list** コマンドでパスやバージョンを確認してください。

⚠ 重要なお知らせ

Python 3.17以降、Windows向けの従来の.exeインストーラーは廃止され、**Python Install Manager (py.exe)** を利用したインストールが必須となる予定です。これからPythonを始める方は、Python Install Managerの導入を強くお勧めします。

2.1.2 Python Install Managerのダウンロードとインストール手順📖

1. 手順1：古いPy launcher（Pyランチャー）の存在チェック📖

```
```bash
```

```
C:\Users\seraku> where py
```

情報：与えられたパターンのファイルが見つかりませんでした。

と表示される。

または

見つかった場合は以下を実行します。

アンインストール方法：

1. 「設定」→「アプリ」→「インストールされているアプリ」を開く

2. リストから "Python Launcher"というアプリを選択してアンインストール
3. すべての py.exe が削除されたことを確認(本コマンドを再実行し確認)
- ...

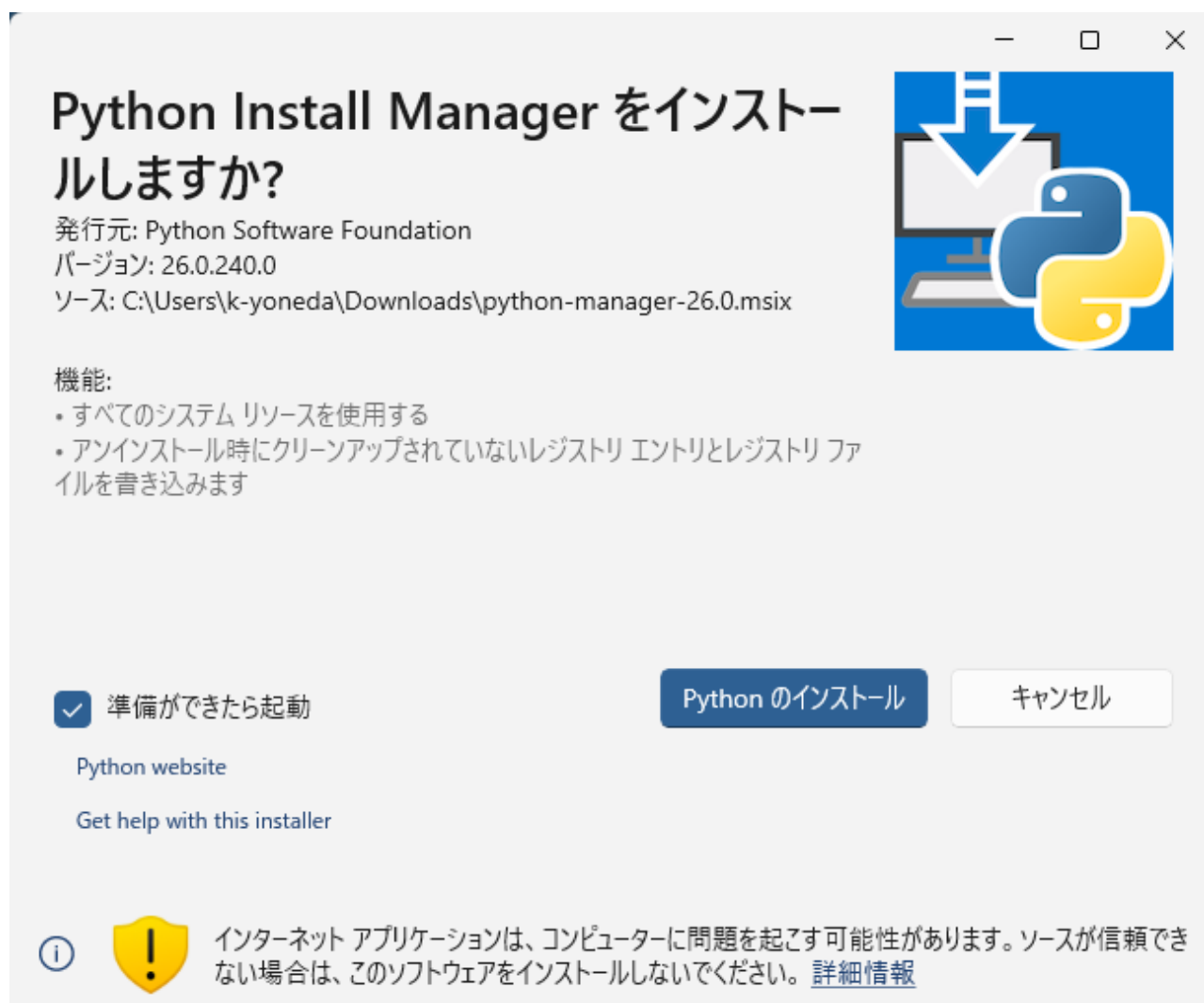
## 2. 手順2：ダウンロード

以下公式サイト of Windows向けダウンロードページにアクセスし、最新のPython Install Managerをダウンロードします。

- [Python for Windows公式ダウンロードページ](#)
- ページ内にある "**Download Python Install Manager**" またはそれに類するリンクから、MSIXファイルをダウンロードしてください。

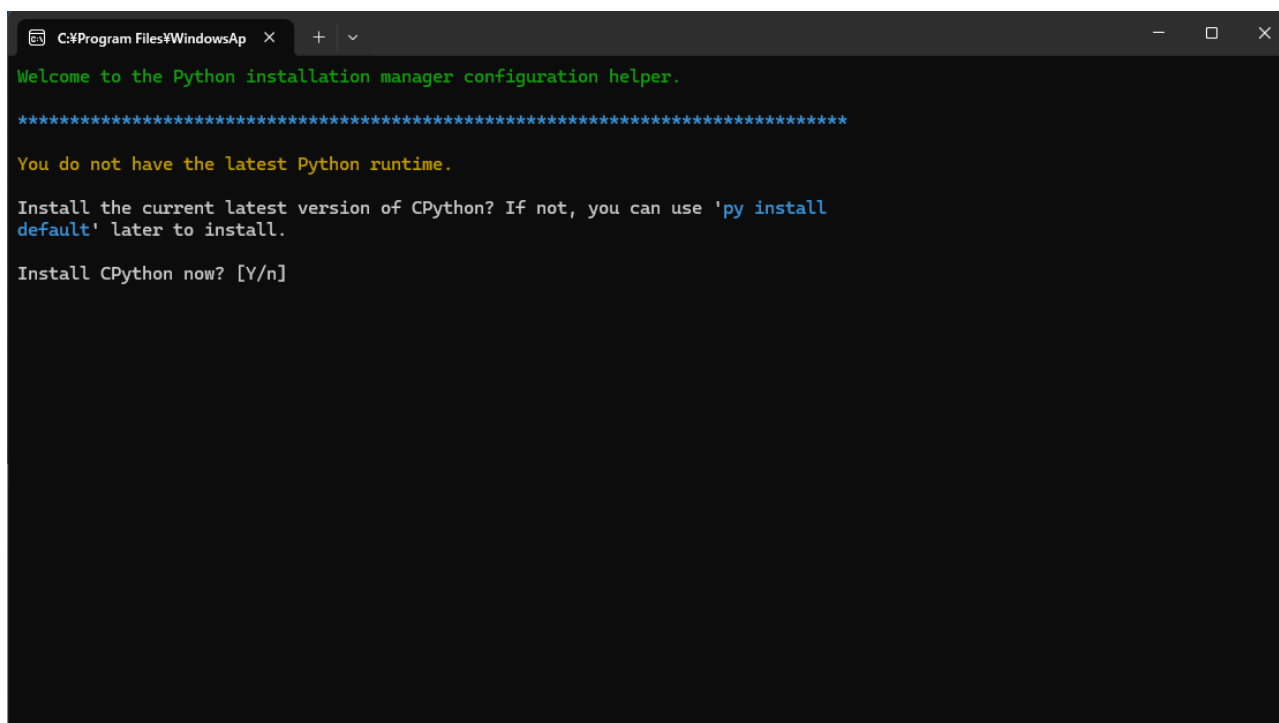
## 3. 手順3：インストール

- ダウンロードしたファイル（例: `python-manager-26.0.msix`）をダブルクリックします。
- インストーラーが起動すると、**ウィザード形式**で複数画面が順に表示されます。表示に従って「次へ」や「インストール」をクリックし、完了させます。



- **ウィザードの各画面で推奨される操作**（バージョンにより画面構成や文言が異なる場合があります）：
- **インストール先**: MSIX版ではユーザー領域（%LOCALAPPDATA%\Programs\Python Manager など）に固定されており、変更できません。

- **PATH への追加・ショートカット**: 「PATH に追加する」「ショートカットを作成する」などのオプションがあれば、\*\*有効（推奨）\*\*にすると、ターミナルから **py** コマンドが使えるようになります。
- **既存の Python / py.exe がある場合**: 「置き換えますか？」などの確認が出た場合は、\*\*置き換え（上書き）\*\*で進めてかまいません。既存の仮想環境に依存している場合は、事前にバックアップやメモを取っておくと安心です。
- インストール完了後、コマンドプロンプトが自動で開き、次のような画面が表示されます。  
**CPython**（C言語で実装された公式のPython本体。頭の「C」はプログラミング言語のCを指します）をインストールするかどうか尋ねられるので、**y** または **Y** を入力して Enter を押すと、最新の Python がインストールされます。**Y と y は同じ意味**（「はい」）で、大文字・小文字のどちらを入力してもかまいません。



#### 4. 手順4：インストール確認

- 上記の CPython インストール案内画面で **y** を入力した場合、**Python 本体のインストールもここで完了しています**。コマンドプロンプトで次を実行し、バージョンが表示されれば正常です。

```
C:\Users\seraku> py --version
Python 3.14.0
```

- **y** を入力しなかった場合や、その画面を閉じてしまった場合は、Python 本体がまだインストールされていません。**2.2 他のバージョンのインストール方法** の「2.2.2 特定バージョンのインストール」に従い、**py install 3** を実行してインストールしてください。
- **py** コマンドが使えるかだけ確認したい場合は、**py** と入力して Enter を押し、ランチャーのヘルプやメッセージが表示されれば Python Install Manager は正常にインストールされています。

## 2.2 他のバージョンのインストール方法

2.1 で最新の Python がすでにインストール済みの方は、この節は不要です。別のバージョン（例：3.13）を追加で入れたい場合や、2.1 で Python 本体を入れていなかった場合に参照してください。

### 2.2.1 コマンドプロンプトを開きます。

- **Windowsの場合:** Windowsの下を検索窓で、**cmd**と入力してください。以下のような画面が表示されます。

注意）以下はログインユーザ名が**seraku**の場合のコマンドプロンプトの初期表示です。ログインユーザ名で読み替えてください。

```
コマンドプロンプト起動直後の画面イメージ
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\seraku>| <-ここに入力します
```

### 2.2.2 特定バージョンのインストール

`py install <version>` で、必要なバージョンを追加インストールできます。2.1 で Python 本体を入れていない場合は、ここで `py install 3` を実行してください。

1. **インストール済みバージョンの確認** 現在インストールされている Python のバージョン一覧を確認します。

```
C:\Users\seraku> py list↵
```

2. **最新版（3系）のインストール** **py install 3** コマンドで、利用可能な最新の Python 3 系をインストールします。

```
C:\Users\seraku> py install 3↵
```

- インストールが成功すると、以下のようなメッセージが表示されます。

```
Python 3.14.0 was installed successfully.
```

※バージョン番号はインストール時点の最新版です。

**特定バージョン（例：3.13）を入れたい場合は**、次のようにバージョン番号を指定します。



```
C:\Users\seraku> py install 3.13
```

3. **インストール後の確認** 再度 **py list** を実行し、新しいバージョンがリストに追加されていることを確認します。

```
C:\Users\seraku> py list
```

- 出力例：

```
-V:3.14 * Python 3.14
```

※ \* は現在デフォルトとして設定されているバージョンを示します。

### 2.2.3 インストール先と既存環境との共存

- py install** でのインストール先: **py install** コマンドでインストールされるPython本体は、通常ユーザープロファイル内の以下の場所に格納されます。

```
C:\Users\<ユーザー名>\AppData\Local\Programs\Python\
├─ Python314\
│ ├── python.exe
│ ├── Scripts\
│ │ └─ pip.exe
│ └─ Lib\site-packages\
└─
```

- 既存の .exe インストーラ版との共存について:** 既に .exe インストーラでPython 3.14などがインストールされている環境で、**py install 3.14** を実行した場合、**既存の環境は上書きされません**。
  - 挙動:** Python Install Managerは、.exe版とは**独立した新しい環境**として、自身が管理するパス（上記のパス）にPythonをインストールします。
  - 結果:** PC内には同じバージョンのPythonが複数共存する状態になります。一つは.exeインストーラ由来のもの、もう一つはPython Install Manager由来のものです。
  - 使い分け:** **py.exe**ランチャーは両方の環境を認識します。**py list**で一覧を確認し、**py -3.14**のようにバージョンを指定することで、どちらの環境を利用するかを明示的に選択できます。
- パスの確認方法:** 実際にどの**python.exe**が使われているかを確認するには、**where**コマンドが便利です。

```
where python
```

- 出力例（複数のPythonが存在する場合）：

```
C:\Users\<ユーザー名>
>\AppData\Local\Programs\Python\Python314\python.exe
C:\Program Files\Python314\python.exe
```

- パスを暗記する必要はありませんが、「どのpython.exeが使われているか」を確認したいときに役立ちます。

## 2.2.4 実行とバージョン確認

- **バージョン確認:** デフォルトに設定されているPythonのバージョンを確認します。

```
C:\Users\<ユーザー名> py --version
Python 3.14.0
```

- **実行方法の例:**

- **py**
  - デフォルト設定のPythonで、対話モード（REPL）を起動します。
- **py script.py**
  - 指定したPythonスクリプト（**script.py**は実際のファイル名に置き換えます）を実行します。
- **py -3.14**
  - インストール済みの特定バージョン（この例では3.14）を指定して、対話モードを起動します。
- **py -3.14 script.py**
  - 特定バージョンを指定してスクリプトを実行します。

---

## 3. NewtonX ADK開発環境の構築手順 📖

### 3.1 プロジェクトディレクトリの準備 📖

**他のプログラム開発を行いたい場合:** 本章の手順は、プロジェクト名（例: 本文中の **newtonx\_project**）を任意の名前に変更して実施すれば、同様に開発環境を構築できます。NewtonX ADK 以外のプロジェクトでも、フォルダ名や仮想環境名を読み替えて同じ流れで進められます。

#### 3.1.1 コマンドプロンプトを開きます。 📖

- **Windowsの場合:** Windowsの下を検索窓で、**cmd**と入力してください。以下のような画面が表示されます。

注意) 以下はログインユーザ名が**seraku**の場合のコマンドプロンプトの初期表示です。ログインユーザ名で読み替えてください。

```
コマンドプロンプト起動直後の画面イメージ
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\seraku>| <-ここに入力します
```

3.1.2 以下のディレクトリ（フォルダ）を作成し、そこに移動してください。

```
📁 プロジェクト用ディレクトリ作成

C:\Users\seraku> cd Documents↵ # ← ユーザー入力 📁
C:\Users\seraku\Documents> mkdir newtonx_project↵ # ← ユーザー入力 📁
C:\Users\seraku\Documents> cd newtonx_project↵ # ← ユーザー入力 📁

現在のディレクトリ
C:\Users\seraku\Documents\newtonx_project>
```

### 3.2 仮想環境の作成 📁

```
NewtonX ADK専用の仮想環境を作成 📁
現在のディレクトリをプロンプトで確認
C:\Users\seraku\Documents\newtonx_project>
C:\Users\seraku\Documents\newtonx_project> py -m venv myvenv↵ # ← ユーザー入力 📁

今回は仮想環境名 `myvenv` で作成しています。
他によくある例です。（これは入力しないでください）
py -m venv venv # venvという名前の仮想環境を作成
py -m venv .venv # .venvという名前の仮想環境を作成
```

**注意:** 複数のPythonバージョンをインストールしている場合は、`py -3.14 -m venv myvenv` のようにバージョンを指定してください。

### 3.3 仮想環境の有効化 📁

```
現在のディレクトリをプロンプトで確認
C:\Users\seraku\Documents\newtonx_project>
仮想環境を有効化します。
C:\Users\seraku\Documents\newtonx_project> .\myvenv\Scripts\activate↵ # ← ユーザー入力 📁
仮想環境が有効になると、プロンプトの先頭部分の表示が変わります。
(myvenv) C:\Users\seraku\Documents\newtonx_project>
↑↑↑↑↑↑↑↑ コマンドラインの先頭に仮想環境名(myvenv)が表示されているか確認する。 📁

有効化の確認（インストール時のPython.exeではなく、venv内のPython.exeが参照されている）
where python
出力例: C:\path\to\newtonx_project\myvenv\Scripts\python.exe
```

```
macOS / Linux
source myenv/bin/activate

有効化の確認 (venv内のPythonが参照されている)
which python
出力例: /path/to/newtonx_project/myenv/bin/python
```

### 重要 (以降のコマンド実行について) 📖 :

- `pip install` や `python/py` 実行時、仮想環境が有効化されていることを確認してください。📖 ⇒ コマンドラインの先頭に仮想環境名(myenv)が表示されているか確認する。
- 仮想環境が有効な場合、`python` コマンドでも仮想環境内のPythonが使用されます。
- VSCodeで新しい統合ターミナルを開くたび、自動有効化が無い場合は毎回 `activate` (Windows) または `source myenv/bin/activate` (macOS/Linux) が必要です。VSCode以外でターミナルを開き、各種コマンドを実行する場合、本環境での操作の場合は、`activate`されていることが必要です。📖
- VSCodeでの仮想環境の扱いと自動有効化に関する注意点は 4.5.3 を参照してください。

### 仮想環境が有効かどうかの確認方法:

- コマンドプロンプトの先頭に **(myenv)** が表示される。※重要📖
- `where python` で仮想環境のパスが表示される。

注: PowerShellでactivateする場合は、以下の手順を先に実施してください。

- PowerShellを管理者権限で実施
- 以下のコマンドを実行

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser -Force
```

## 3.4 NewtonX ADKのインストール 📖

### 3.4.1 NewtonX ADKの取得 📖

NewtonX ADKの利用は、以下のURLのフォームから利用申請をしてください。フォーム送信後に、ADK本体をダウンロードできるURLを表示します。それをクリックするとダウンロード可能な画面に遷移します。万が一、遷移前にURLの表示を閉じてしまった場合は、再度フォームで申請してください。

**申請フォームURL** 📖 : [https://forms.office.com/Pages/ResponsePage.aspx?id=M8Cw5UMcMECWs-ggP\\_dw0SiQ14m9dWJHjxPWhL9GSdZUREI4Q0ZUWlhOV0cwTIYxRU04UzVZQ0hZSC4u](https://forms.office.com/Pages/ResponsePage.aspx?id=M8Cw5UMcMECWs-ggP_dw0SiQ14m9dWJHjxPWhL9GSdZUREI4Q0ZUWlhOV0cwTIYxRU04UzVZQ0hZSC4u)

### 3.4.2 ADKの解凍と配置 📖

ダウンロードしたzipファイルを解凍すると、newtonx\_adkというフォルダ名の中にadk一式が復元されます。そのフォルダごと、newtonx\_projectの下に配置し、以下のインストールをおこなってください。

```

C:\Users\<ユーザー名>\Documents(にnewtonx_projectを作成した場合)
├─ newtonx_project\
│ └─ newtonx_adk # newtonx_adkをダウンロードして解凍したものをその
 まま配置する。
├─ newtonx_adk-0.10.5-py3-none-any.whl # 取得したADKのバ
 ージョンによって、名称が異なる場合があります。
├─ adk_examples
├─ skills
├─ tools
└─ docs

```

### 3.4.3 ADKの仮想環境へのインストール（超重要）

- 上記に配置したnewtonx\_adkを、仮想環境にインストールします。

必ずコマンドプロンプトの先頭に仮想環境名(myvenv)が表示されている状態で実施してください。



# 現在のディレクトリと仮想環境が有効であることをプロンプトで確認)

```
(myvenv) C:\Users\seraku\Documents\newtonx_project>
```

```
(myvenv) C:\Users\seraku\Documents\newtonx_project>cd newtonx_adk # ← ユー
ザー入力 
```

# 現在のディレクトリ確認)

```
(myvenv) C:\Users\seraku\Documents\newtonx_project\newtonx_adk>
```

# ADKを仮想環境にインストール (newtonx\_adkフォルダ内の.whlファイル名を指定。バージョンによりファイル名が異なります)

```
(myvenv) C:\Users\seraku\Documents\newtonx_project\newtonx_adk> pip
install newtonx_adk-0.10.5-py3-none-any.whl # ← ユーザー入力 
```

#### インストール後の確認:

# ADKが正しくインストールされたか確認 (仮想環境を有効化した状態で実行してください)

```
python -c "import newtonx_adk; print('NewtonX ADK installed
successfully')"
```

### 3.5 configuration fileの準備 (VSCodeからアプリ実行のための事前準備)

VSCodeという開発環境を後ほどインストールします。そこで作成したPythonアプリを実行するのに必要なファイルを事前に準備しておきます。

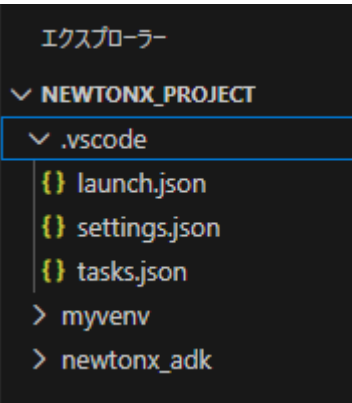
理由) VSCodeから実行する場合、コマンドラインから実行するのとは異なり引数を渡すことができません。そのため、あらかじめ、実行時に渡す引数などを指定したファイルを作成し指定位置に配置します。

VSCodeに読み込んだフォルダ直下に **.vscode** というフォルダを作成し、次の手順でファイルを配置します。

1) エクスプローラでドキュメントを表示し、**newtonx\_project**をクリックして移動します。その下に右クリックで新規作成でフォルダを選んで、**.vscode**という名前のフォルダを作成し、配布している3つのjsonファイルをその

下にコピーしてください。

2) NewtonX-ADKと一緒に配布されているJson.zipを解凍し、Windows/macフォルダのwindowsの下にtasks.json/launch.json/settings.jsonというファイルがあります。これを1) で作成したフォルダ下に配置してください。



■配置する3つのJsonファイル

ファイル名	役割	正式な呼び方
tasks.json	ビルドやスクリプトの自動実行設定	Task configuration file
launch.json	デバッグ構成（実行・ブレイクポイントなど）	Debug configuration file
settings.json	起動時に仮想環境の自動化指定	Settings File (ユーザー設定ファイル)

※入手方法：tasks.json、launch.jsonおよびsettings.jsonはADK内には同梱しておりませんが、ADKダウンロードサイトで別途ダウンロードが可能です。ご自身で.vscodeディレクトリを作成し配置してください。📁

## 参考情報

以下のセクションは、仮想環境「venv」に関する詳細な説明です。第3章の手順を理解するための参考としてご活用ください。venvの基本概念、必要性、基本的なコマンド操作について包括的に説明しています。

### 1. 「venv」とは

venvは、Pythonに組み込まれている仮想環境を作成するためのツールです。Python 3.3以降で標準搭載されており、プロジェクトごとに独立したPython環境を構築できます。**この章のコマンドは第3章で使います。概要を理解してください。**

### 2. 仮想環境の必要性

NewtonX ADK開発においてvenvを使用する理由：

- **依存関係の分離**: NewtonX ADK専用のパッケージ環境を作成。
- **システム環境の保護**: システムのPython環境を汚染しない。
- **再現性の確保**: 同じ環境を他の開発者と共有可能。
- **バージョン管理**: 特定のPythonバージョンを固定してプロジェクトを進められる。

### 3. 基本的なvenvコマンド

#### 3.1 環境の有効化・無効化

```
仮想環境の有効化（すでに実行済みです）
.\myvenv\Scripts\activate

仮想環境の無効化（仮想環境を終了させたいときに入力してください）
deactivate
```

#### 3.2 パッケージ管理

```
パッケージのインストール
pip install package_name

特定バージョンのインストール
pip install package_name==1.0.0

インストール済みパッケージの確認
pip list

NewtonX ADKの確認
pip show newtonx_adk
```

#### 3.3 依存関係の管理（サンプルアプリ実行の際に再構築が必要です）

```
現在の環境の依存関係を保存
pip freeze > requirements.txt

依存関係から環境を再構築
pip install -r requirements.txt
```

### 3.4 仮想環境にnewtonx\_adkをインストールした後のイメージ

なぜターミナル操作の際にactivateが必要なのかは以下を参考にしてください。

1) 最初にPythonをインストールした直後

```
C:\Users\<ユーザー名>\AppData\Local\Programs\Python\
├── Python314\ # Python ver3.14 (←今回はこれ)
│ ├── python.exe # 実行ファイル ver3.14
│ ├── pip.exe # パッケージ管理ツール ver3.14
│ └── Lib\
│ └── site-packages\ # pip.exe ver3.14でインストールされたライ
ブラリを格納
│
│ ※ここにはnewtonx_adkはインストールされていない

```

2) 仮想環境を作成して、newtonx\_adkをインストールした直後以下が追加される

```
C:\Users\<ユーザー名>\Documents\newtonx_project\
├── myenv\ # 仮想環境フォルダ
│ ├── Scripts\
│ │ ├── python.exe(*) # 仮想環境専用のPython実行ファイル (リンクまた
はコピー)
│ │ ├── pip.exe(*) # 仮想環境専用のpip (リンクまたはコピー)
│ │ └── activate # 仮想環境を有効化するためのスクリプト
│ (Windows用)
│ ├── Lib\
│ │ ├── site-packages\(*) # 仮想環境専用のライブラリ格納フォルダ
│ │ └── newtonx_adk # 仮想環境でインストールされたライブラリ
│ └── pyenv.config # 仮想環境設定ファイル

```

myenvの仮想環境がactivateした場合、python.exe(\*)が実行される (パスの先頭に追加される)

このPython.exe(\*)はpip.exe(\*)でインストールされたライブラリは site-packages\(\*)の下を

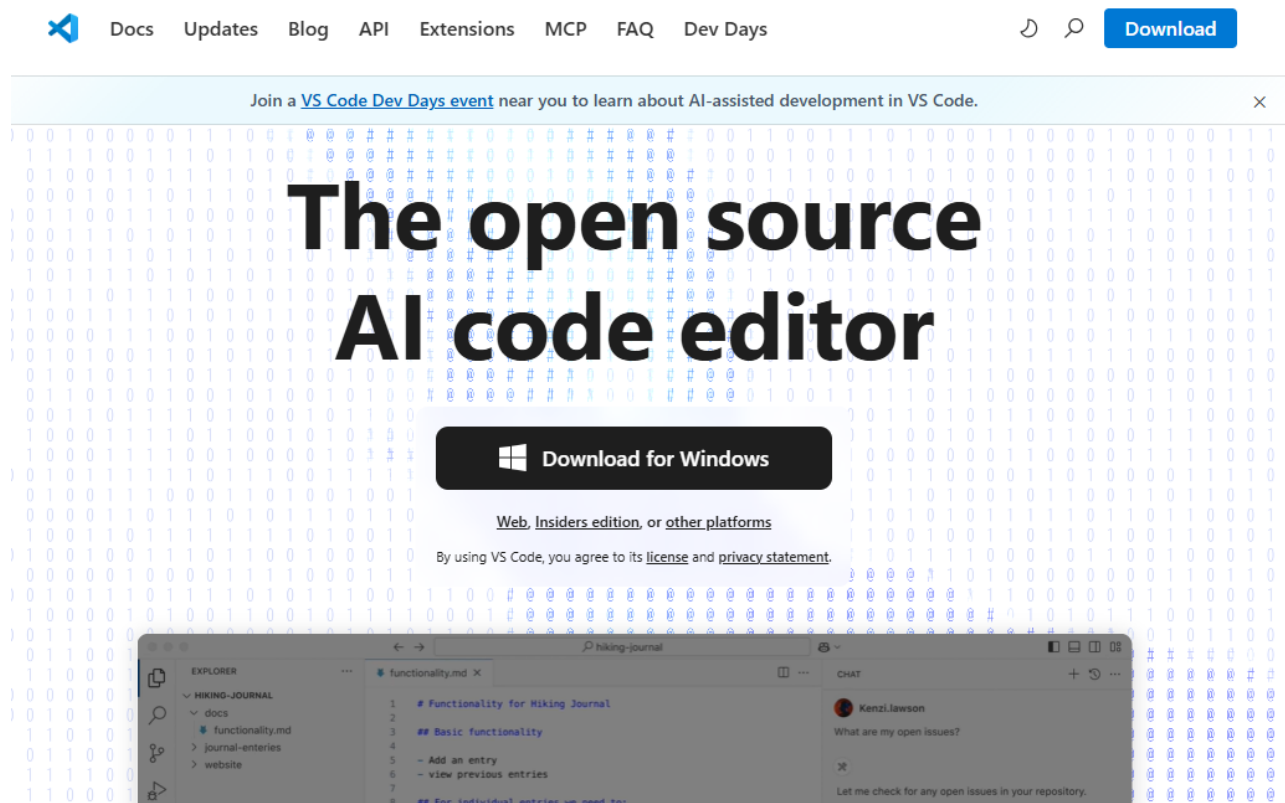
探しにいきます。すべて仮想環境の中に必要なものをコピーして保管している状態です。



## 4. VSCodeのインストールとPython開発環境設定 📖

### 4.1 VSCodeのダウンロード 📖

1. [Visual Studio Code公式サイト](#)にアクセスします。
2. 「Download for Windows」をクリックします。以下はダウンロード画面のスクリーンショットです：

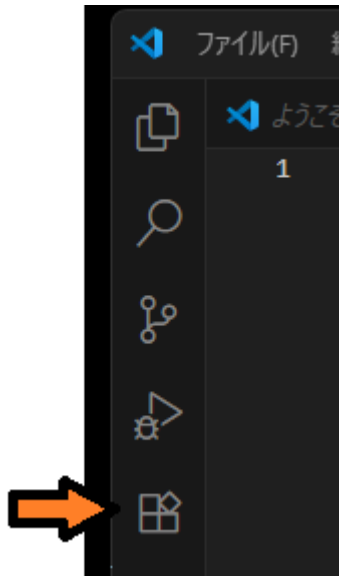


### 4.2 VSCodeのインストール 📖

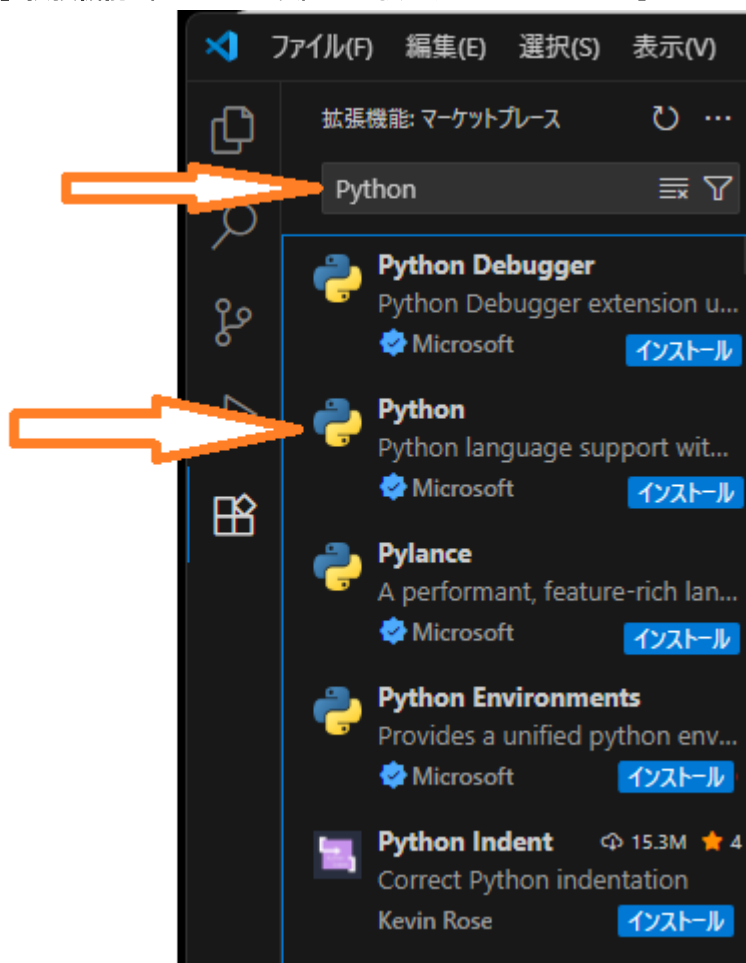
1. ダウンロードしたインストーラー（.exe）を実行します。
2. インストール中に以下を確認してください：
  - 「Add to PATH」にチェックを入れる（Windowsの場合）。
  - 必要に応じて追加のタスク（ショートカット作成など）を選択します。
3. インストールが完了したら、デスクトップやアプリケーションフォルダからVSCodeを起動します。

## 4.3 Python拡張機能のインストール 📦

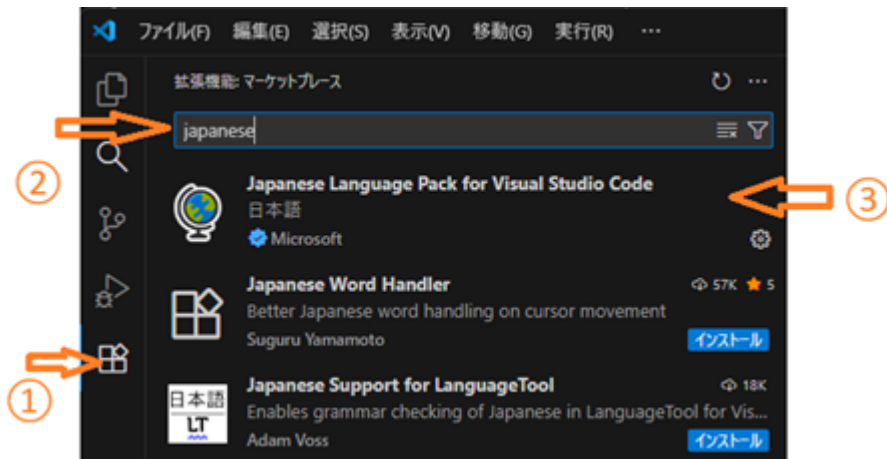
1. **VSCode**を起動します。
2. 左側のアクティビティバーから「拡張機能」アイコン（四角いアイコン）をクリックします。



3. 検索ボックスに「Python」と入力します。
4. 「**Python**」拡張機能（Microsoft製）を選択し、「インストール」をクリックします。

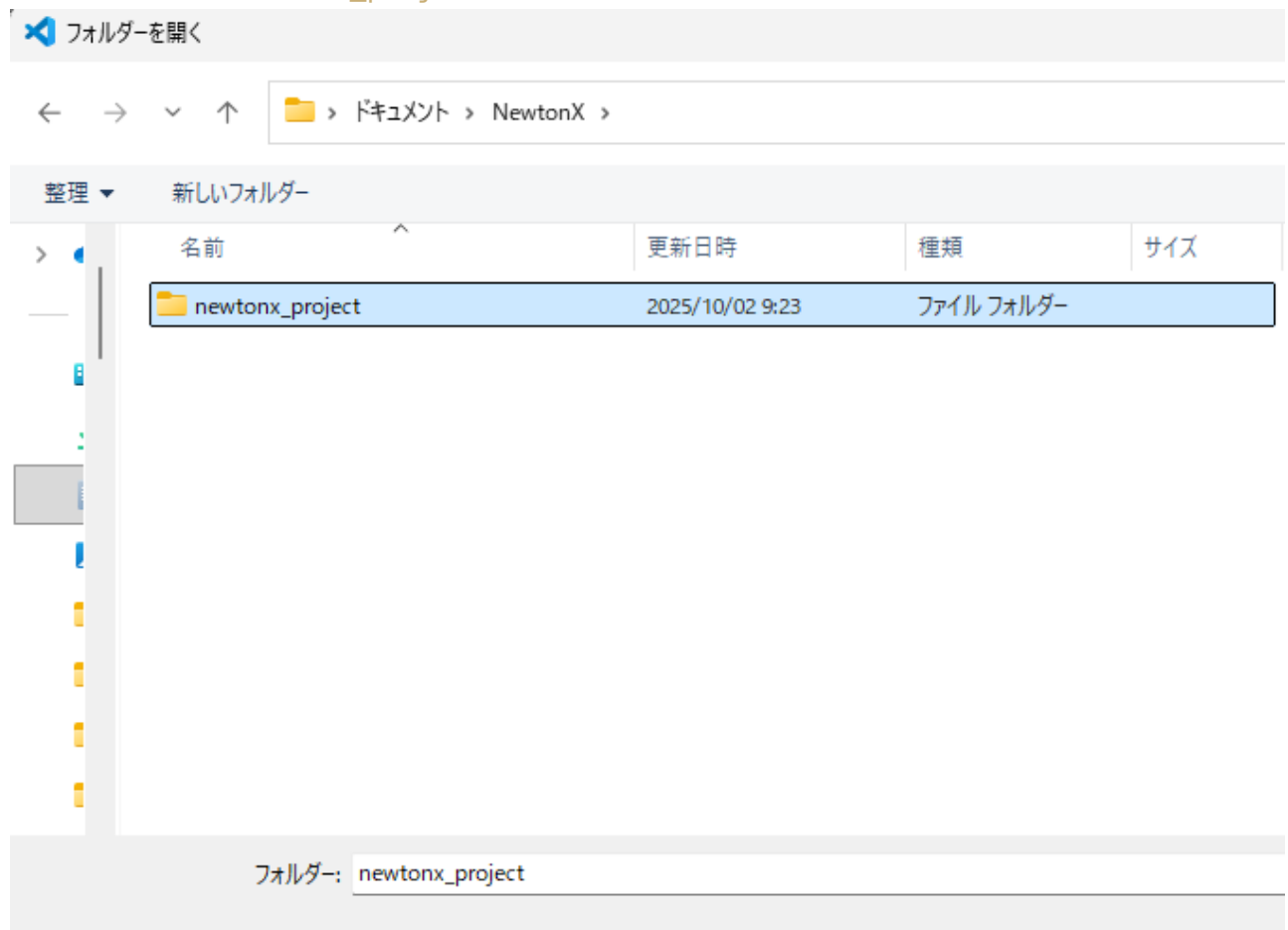


**重要:** Python拡張機能は、VSCodeでPython開発を行うために必須です。なお、日本語化は必要に応じて行ってください。拡張機能の検索窓に「Japanese」と入力し、地球儀マークの「Japanese Language Pack for Visual Studio Code」をインストールしてください。



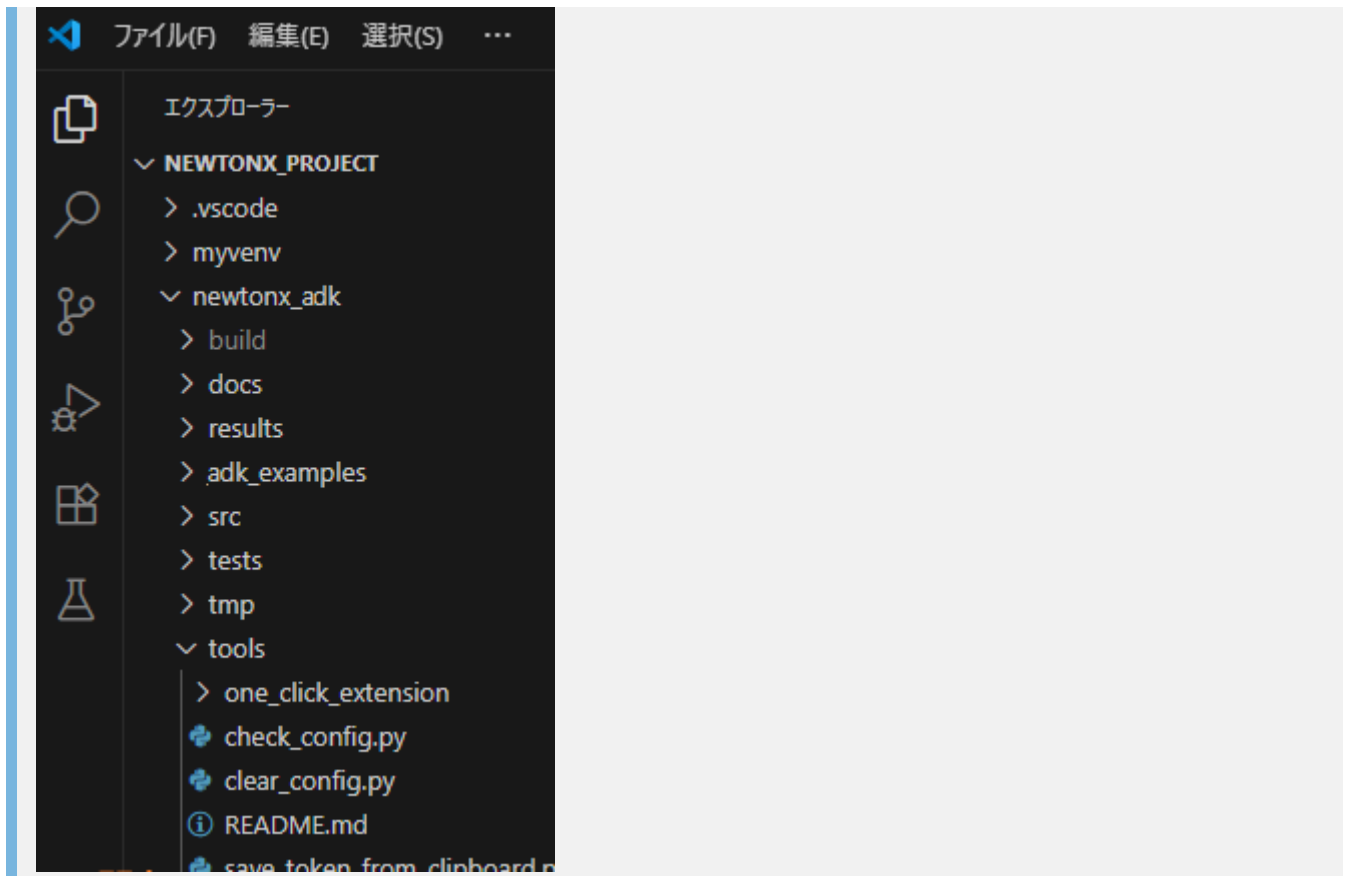
#### 4.4 プロジェクトフォルダの開き方

1. VSCodeのメニューバーから「ファイル」→「フォルダーを開く」を選択します。
2. 第3章で作成した\*\*newtonx\_project\*\*フォルダ\*\*を選択します。



3. 「フォルダの選択」をクリックしてプロジェクトを開きます。
4. 「ようこそ」のtabは✕で閉じてください。

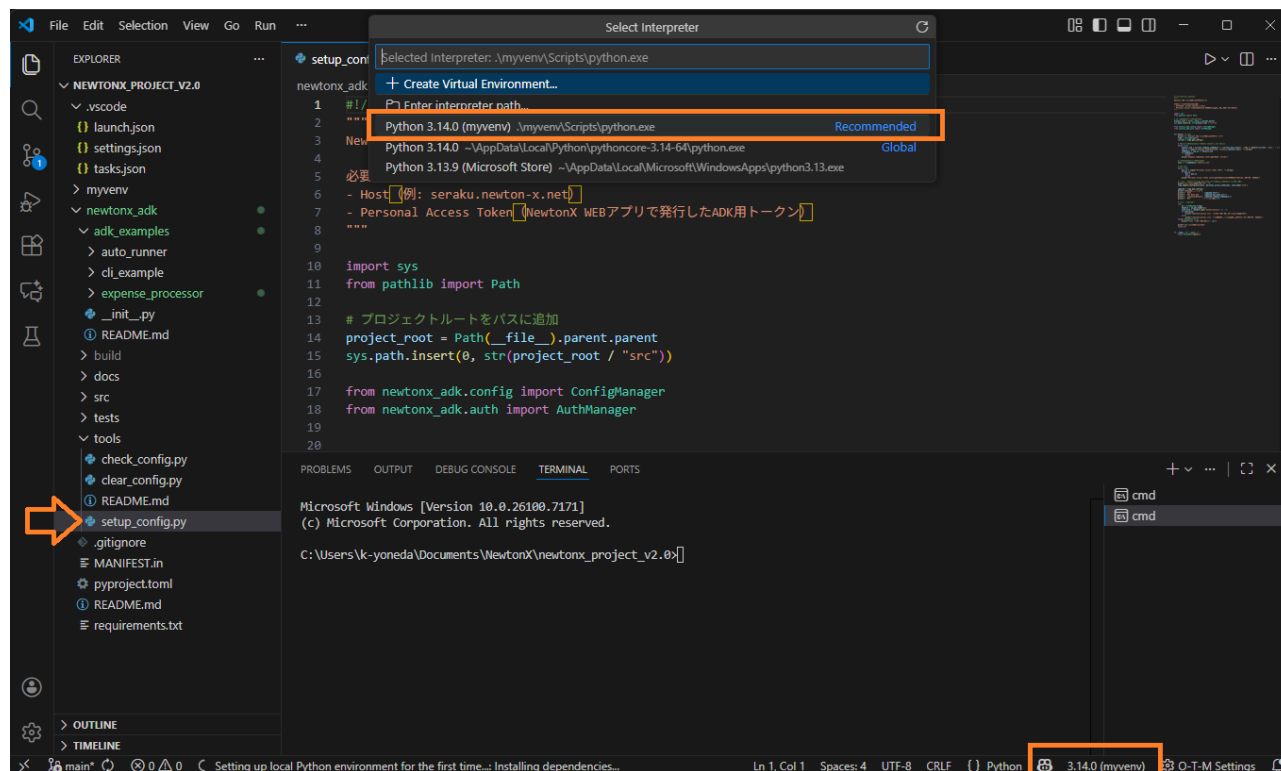
**確認:** VSCodeのエクスプローラー（左側）にnewtonx\_projectフォルダの内容が表示されます。最上位のフォルダ名はすべて大文字で表示されます。newtonx\_project ⇒ NEWTONX\_PROJECT



## 4.5 仮想環境の設定 📖

### 4.5.1 Pythonインタープリターの選択 📖

1. **Pythonファイルを作成または開く** ことで、VSCodeの下部ステータスバーにPythonバージョンが表示されます。
  - 例: `newtonx_adk\tools\setup_config.py` ファイルをクリックしてコードを表示します。  
**`newtonx_adk\tools\setup_config.py`は認証プログラムですので後ほど使用します。**
2. 下部ステータスバーに表示されたPythonバージョンをクリックします。
3. 「Pythonインタープリターを選択してください」が表示されたら、**Python 3.14.0(myenv)...**を選択します（インストールしたバージョンの(myenv)を選択）。

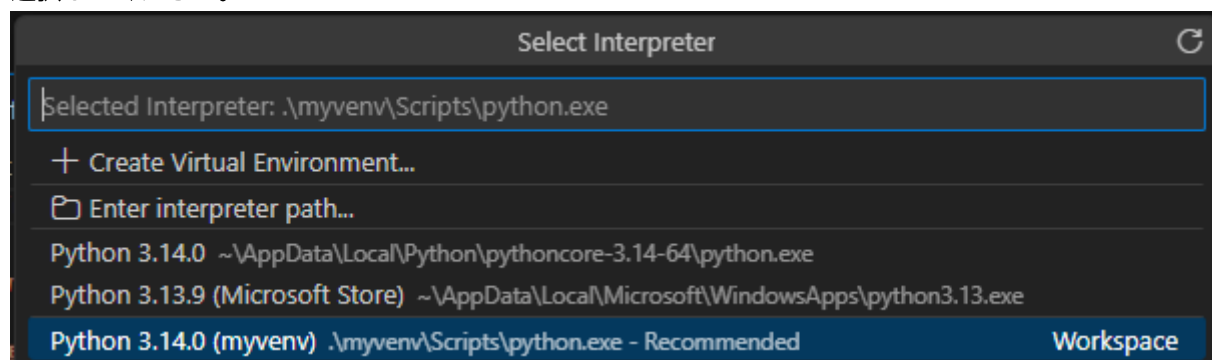


- パス例: `.\myvenv\Scripts\python.exe`
- macOSの例: `./myvenv/bin/python`

**注意:** フォルダを開いただけではPythonバージョンは表示されません。Pythonファイル（拡張子.pyのファイル）を作成または開く必要があります。また、それでも表示されていない場合は、Ctrl + Shift + P を押し、入力欄に「select interpreter」と入力して以下の画面を表示してください。Ctrl + Shift + P はよく使いますので覚えておいてください。



クリックして下の画面でPython `**3.14.0(myvenv)...**` を選択してください。バージョンはインストールしたものを選択してください。



(macOSのショートカット)

- コマンドパレット: Cmd + Shift + P

#### 4.5.2 ターミナルのデフォルトをコマンドプロンプトに設定

パワーシェルが起動される場合、管理者モードでの操作が必要になることがありますので、本ガイドでは、コマンドプロンプトをデフォルトとして説明します。

1. コマンドパレットを開きます (Ctrl + Shift + P) 。 
2. 「Terminal: Select Default Profile」 (日本語: 「ターミナル: 既定のプロファイルの選択」) を選択します。  

3. 一覧から「Command Prompt」を選択します。 
4. 既存のターミナルをすべて閉じ、VSCodeで新しいターミナルを開くとコマンドプロンプトで起動します。  


補足 (設定画面からの変更) :

- メニュー「ファイル」→「ユーザー設定」→「設定」 (Ctrl + ,) を開き、検索欄に「default profile」と入力。
- 「ターミナル > 統合 > 既定のプロファイル (Windows)」で「Command Prompt」を選択します。

補足 (settings.jsonで直接設定) :

```
"terminal.integrated.defaultProfile.windows": "Command Prompt",
"terminal.integrated.profiles.windows": {
 "Command Prompt": {
 "path": "C:\\\\Windows\\\\System32\\\\cmd.exe"
 }
}
```

注意: 既存のターミナルには反映されません。設定後に新しい統合ターミナルを開いてください。

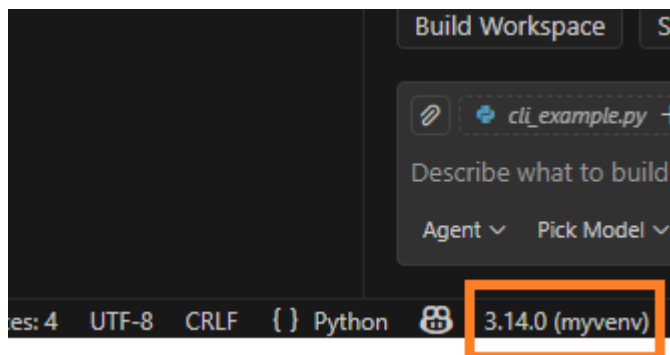
(macOSの場合の設定例)

- コマンドパレット (Cmd + Shift + P) で「Terminal: Select Default Profile」を選択し、「zsh」または「bash」を選択。
- settings.jsonで直接設定する場合:

```
"terminal.integrated.defaultProfile.osx": "zsh",
"terminal.integrated.profiles.osx": {
 "zsh": { "path": "/bin/zsh" },
 "bash": { "path": "/bin/bash" }
}
```

#### 4.5.3 仮想環境の確認

1. VSCodeの下部ステータスバーで、選択されたPythonインタープリターが3.14.0(myvenv)になっていることを確認します。 バージョンはインストールしたものになっていること。 



または

- ターミナル（「表示」 → 「ターミナル」）を開き、以下のコマンドで仮想環境が有効になっていることを確認します：

#### where python

```
出力例（仮想環境が有効な場合）：パスの順に表示されます。
C:\path\to\newtonx_project\myvenv\Scripts\python.exe
C:\Users\<ユーザー名>\AppData\Local\Programs\Python\Python314\python.exe
```

macOSの場合:

#### which python

```
例（仮想環境が有効な場合）：
/path/to/newtonx_project/myvenv/bin/python
```

#### 確認ポイント:

- 最初に表示されるPythonが実際に使用されるPythonです（PATHの優先順位）
- 仮想環境が有効な場合、`myvenv\Scripts\python.exe`が最初に表示されることを確認してください
- 2つ目に表示されるのは標準のPythonです。

注意: VSCodeで新しい統合ターミナルを開いた場合、仮想環境が自動で有効化されないことがあります。自動有効化が効かない場合は、毎回 `activate` (Windows) または `source myvenv/bin/activate` (macOS/Linux) を実行してください (3.3参照)。

## 5. 認証プログラムの実行📖

### 5.1 アクセストークンの取得準備📖

#### 5.1.1 NewtonXの起動📖

左端のバーからアカウントボタンを押し、その右のアクセストークンを選択します。

チャット

アシスタント

部署

アカウント

お知らせ

...

その他

アカウント情報

共有チャット管理

申請一覧

アクセストークン

テンプレート

アカウント名

米田 憲司

メールアドレス

k-yoneda@seraku.co.jp

所属部署

所属している部署はありません

トークン利用量 (次回リセット日: 2026年03月01日)

GPT-5 0 キロトークン

GPT-5.2 451 キロトークン

GPT-5 mini 10394 キロトークン

GPT-4o 1619 キロトークン

#### 5.1.2 アクセストークン取得1📖

右側の発行ボタンをクリックしてください。

チャット

アシスタント

部署

アカウント

お知らせ

...

その他

アカウント情報

共有チャット管理

申請一覧

アクセストークン

テンプレート

NewtonX ADK Token 発行

発行

NewtonX ADKを使用するためのPersonal Access Tokenを発行します。  
トークンは発行時にしか表示されません。

一括削除

トークンを検索...

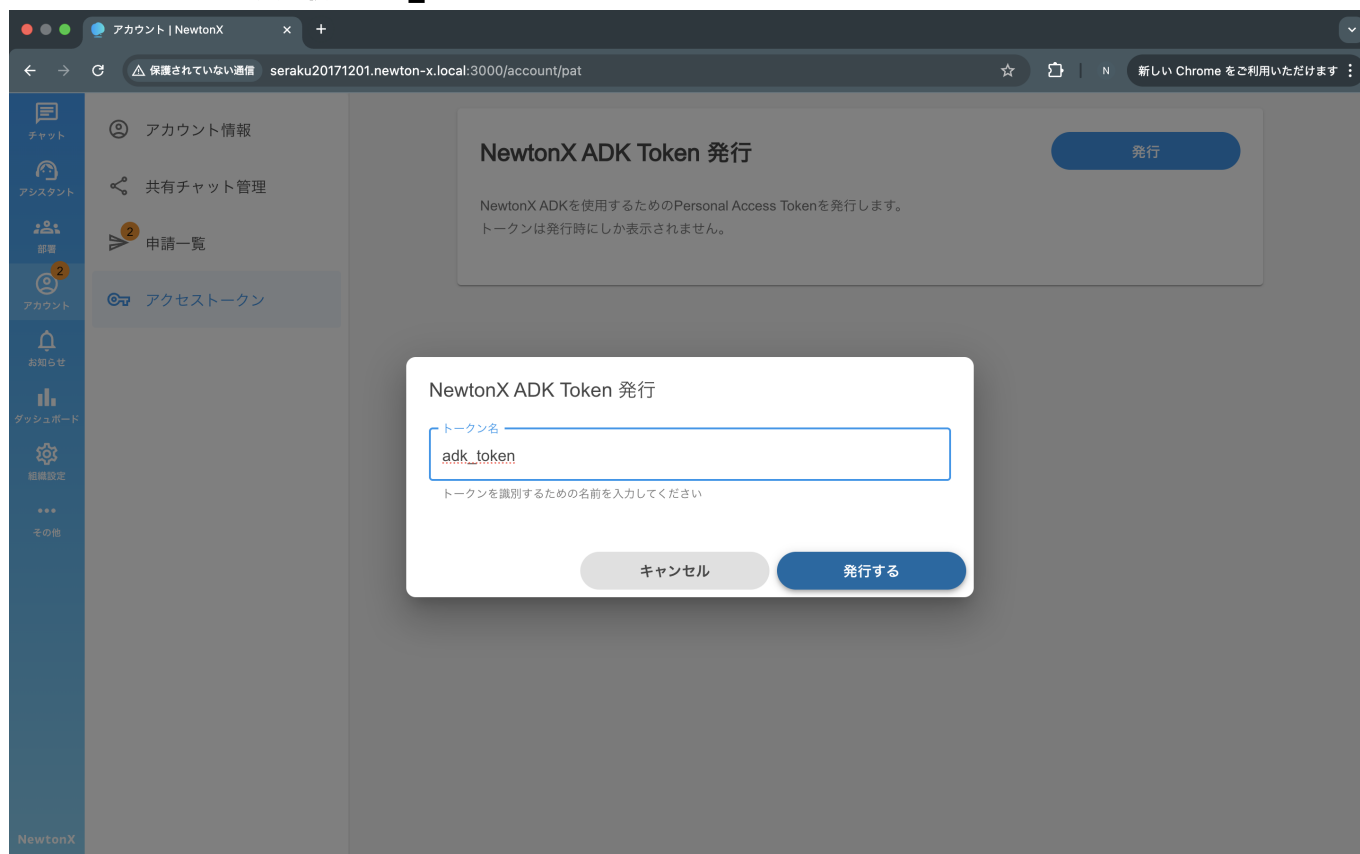
<input type="checkbox"/>	トークン名	有効期限	
<input type="checkbox"/>	test	2026/05/20	<a href="#">リフレッシュ</a> <a href="#">削除</a>
<input type="checkbox"/>	test	2026/05/18	<a href="#">リフレッシュ</a> <a href="#">削除</a>
<input type="checkbox"/>	test	2026/05/13	<a href="#">リフレッシュ</a> <a href="#">削除</a>
<input type="checkbox"/>	test	2026/05/05	<a href="#">リフレッシュ</a> <a href="#">削除</a>
<input type="checkbox"/>	test	2026/05/05	<a href="#">リフレッシュ</a> <a href="#">削除</a>

ページあたりの行数: 5 1~5 / 35

#### 5.1.3 アクセストークン取得2📖

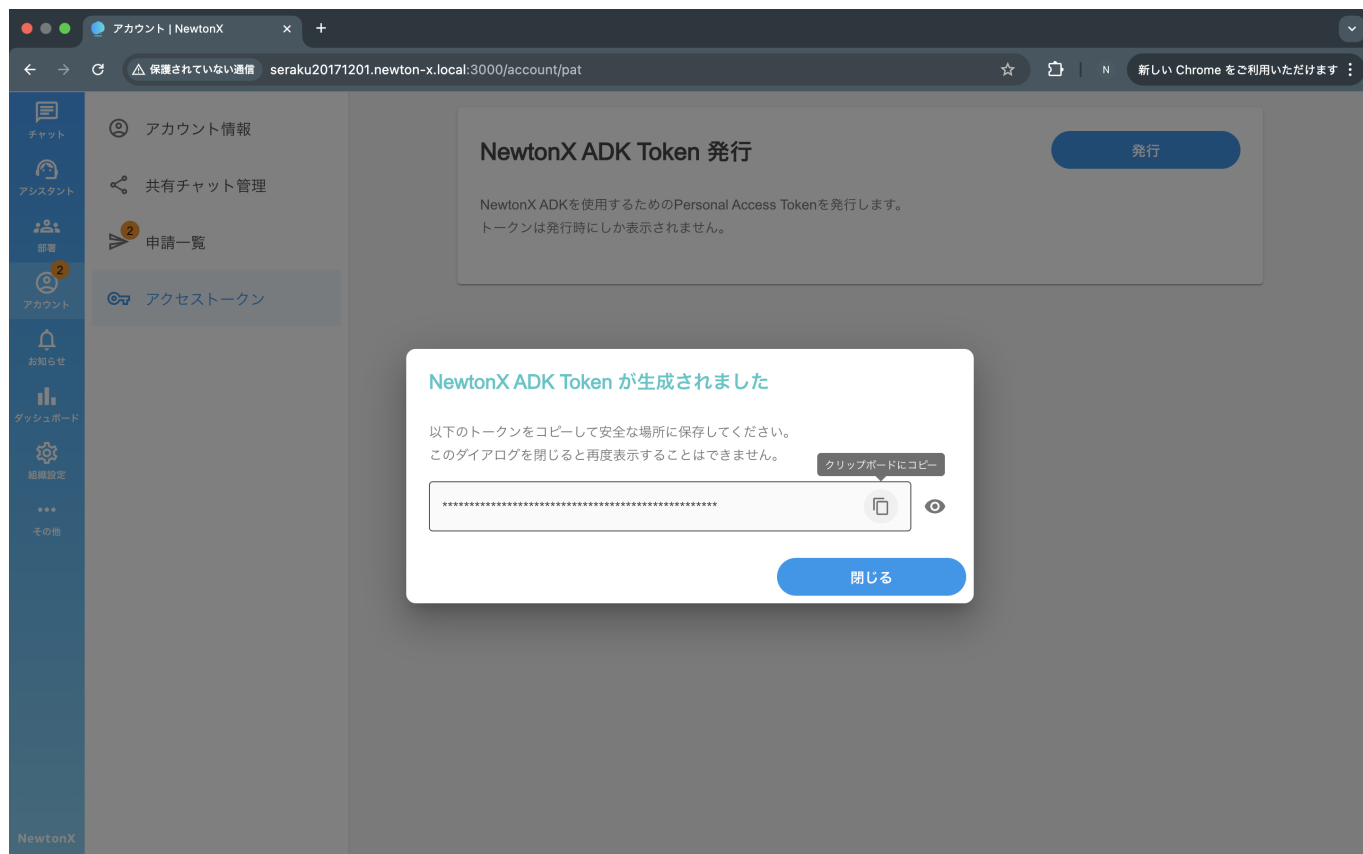


ダイアログが開くのでトークン名を入力して、発行ボタンをクリックします。トークン名は必須入力項目です。名前はなんでもOKです。**入力例：adk\_token**



#### 5.1.4 アクセストークン取得3 📄

発行されるとトークンがマスクされた状態で表示されます。目のアイコンで表示・非表示切り替えます。認証プログラム実行後、ターミナルに貼り付ける直前に **[クリップボードにコピー]** を押してください。 **重要: この画面に表示されるアクセストークンを後ほど使用しますので、本画面は閉じないでください。一旦VSCodeに戻り、次へ進んでください。**

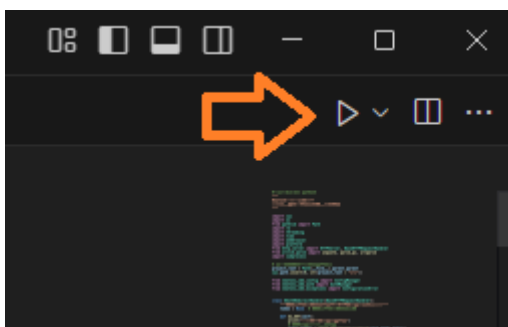


## 5.2 認証プログラムの実行方法📖

- 方法1または方法2のどちらかで認証プログラムを実行してください。

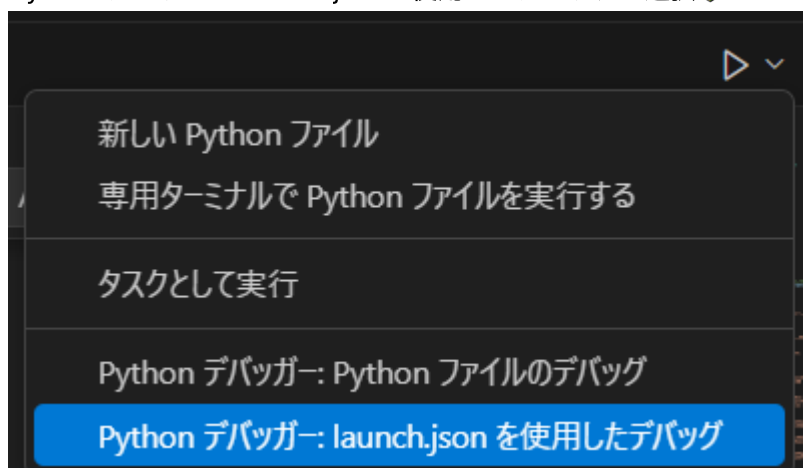
**方法1: VSCodeのデバッグ実行ボタン** ←今回はこちらの方法で実行します。

1. **newtonx\_adk\tools**の下**のsetup\_config.py**をクリックして表示します。**4.5.1 Pythonインタープリター**の**選択**時に表示済であれば、そのコードが最前面に来るようにしてください。表示されていない場合は、4.5.1の項を参照のこと

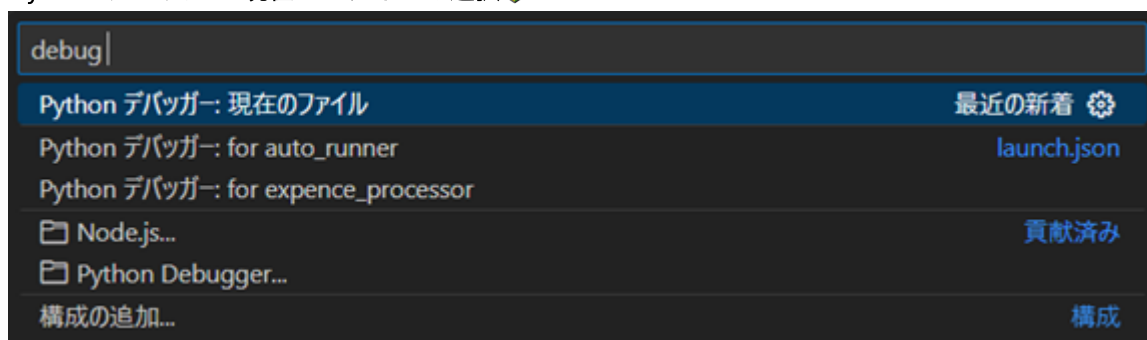


2. 右上の「▶」ボタンをクリックします。📖

### 3. Python デバッガー:launch.jsonを使用したデバッグを選択



### 4. Python デバッガー:現在のファイルを選択



### 5. 認証プログラムが動作します。（→5.3 認証プログラム実行のイメージと入力例へ）

#### 方法2: ターミナルから実行する場合

1. VSCodeのターミナルを開きます（Ctrl + `）
2. 以下のコマンドを実行します：（→5.3 認証プログラム実行のイメージと入力例へ）

```
#必ずコマンドプロンプトの先頭に仮想環境名(myenv)が表示されている状態で実施してください。出来ていない場合は、3.3 仮想環境の有効化を参照して有効にしてください。
(myenv) C:\Users\seraku\Documents\newtonx_project>
(myenv) C:\Users\seraku\Documents\newtonx_project> cd newtonx_adk\tools↵
← ユーザー入力
(myenv) C:\Users\seraku\Documents\newtonx_project\newtonx_adk\tools> py
setup_config.py↵ # ← ユーザー入力
→→ 5.3 認証プログラム実行のイメージと入力例へ
```

macOSの場合:

```
(myenv) cd newtonx_adk/tools↵
(myenv) python3 setup_config.py↵ (→5.3 認証プログラム実行のイメージと入力例へ)
```

注意: 実行前に仮想環境を有効化してください（例: Windowsは `.\myenv\Scripts\activate`、macOSは `source myenv/bin/activate`）。

## 5.3 認証プログラム実行のイメージと入力例

以下の説明をよく読んでから、2つの入力項目を入力ください。

■入力項目1: (要注意!!) 入力項目は以下を参照して間違わないようにしてください (間違えたら再実行してください)

=== NewtonX ADK セットアップ (PAT) ===  
Host (例:seraku) [seraku]:seraku-ex (以下を参照し入力してください)

1. セラク内勤者: "seraku"
2. セラク常駐者: "seraku-ex"
3. セラクビジネスソリューションズ: "bs-seraku"
4. AND Think: "andthink"

本入力を間違えると、実行時に認証エラーとなりますのでご注意ください。普段NewtonXをお使いの際のURLの以下のサブドメインの部分を入力いただきます。https://<サブドメイン>.newton-x.net/login 例:

https://seraku.newton-x.net/login → serakuを入力 次回以降[]内に[seraku]などのように表示されていて、それが正しければEnterだけ入力でも可 間違えた場合、再度、本ステップを実行してください。

### ■入力項目2

Access Token (PAT):  
57298 |\*\\*\*\*\*\* 以下を参考に貼り付け

**\*\*重要\*\*** 5.1.4 アクセストークン取得3 のNewtonXのウィンドウに戻り、  
[クリップボードにコピー]を押して、こちらに貼り付けてください。

設定を保存しています...

更新された設定: Host : seraku.newton-x.net

API Base URL : https://seraku.newton-x.net/api

CompanySubdomain: seraku

PAT : (設定済み)

Authorization ヘッダーを確認しました。設定は完了です。

## 5.4 認証成功後の確認

C:\Users<ユーザー名>.newtonx の下に以下のconfig.jsonが生成され、次回の認証では、その値がデフォルト値として表示され、入力が不要となります。

- config.json( アクセストークンの情報を含んでいますので、共有しないこと。共有パソコンなどで認証した場合は、使用後に削除するなど、管理に十分に注意してください。 )

- 例: 共有PCでの利用後は `C:\Users\<ユーザー名>\.newtonx\config.json` を削除してください。

macOSの場合:

```
/Users/<ユーザー名>/.newtonx/config.json
```

## 6. サンプルアプリの実行 📖

### 6.1 サンプル実行のための準備 📖

この準備は、ターミナルから実行、VSCodeから実行にかかわらず、実行前に1回必要です。

#### 1. 依存関係のインストール 📖

- VSCodeのターミナル (Ctrl + `) を開きます。すでにターミナルが開いている場合はそこで実施可能です。
- 仮想環境のインタープリターを選択している場合は、**activate**コマンドで仮想環境を有効にしてから以下を実行してください

##### 1) cli\_example.py用の実行に必要なインストール 📖

```
#必ずコマンドプロンプトの先頭に仮想環境名(myvenv)が表示されている状態で実施してください。📖
(myvenv) C:\Users\seraku\Documents\newtonx_project> cd
newtonx_adk\adk_examples\cli_example # ← ユーザー入力📖
(myvenv)
C:\Users\seraku\Documents\newtonx_project\newtonx_adk\adk_examples\cli_example>
(myvenv)
C:\Users\seraku\Documents\newtonx_project\newtonx_adk\adk_examples\cli_example> pip install -r requirements.txt # ← ユーザー入力📖
```

##### 2) expense\_processor\_example.py用の実行に必要なインストール 📖

```
必ずコマンドプロンプトの先頭に仮想環境名(myvenv)が表示されている状態で実施してください。📖
newtonx_projectのディレクトリに一旦もどりましょう。今いる場所で以下を入力してください。
(myvenv) ...> cd C:\Users\seraku\Documents\newtonx_project # ← ユーザー入力📖
newtonx_projectのディレクトリにもどったのを以下で確認
(myvenv) C:\Users\seraku\Documents\newtonx_project>
(myvenv) C:\Users\seraku\Documents\newtonx_project> cd
newtonx_adk\adk_examples\expense_processor # ← ユーザー入力📖
expense_processorのディレクトリに移動したのを確認
(myvenv)
C:\Users\seraku\Documents\newtonx_project\newtonx_adk\adk_examples\expense_processor>
expense_processor_exampleを動作させるのに必要な環境をインストール
(myvenv) pip install -r requirements.txt # ← ユーザー入力📖
```

##### 3) auto\_runner.py用の実行に必要なインストール 📖

```
必ずコマンドプロンプトの先頭に仮想環境名(myvenv)が表示されている状態で実施してください。
newtonx_projectのディレクトリに一旦もどりましょう。今いる場所で以下を入力してください。
(myvenv) ...> cd C:\Users\seraku\Documents\newtonx_project # ← ユーザー入力
newtonx_projectのディレクトリにもどったのを以下で確認
(myvenv) C:\Users\seraku\Documents\newtonx_project>
(myvenv) C:\Users\seraku\Documents\newtonx_project>cd
newtonx_adk\adk_examples\auto_runner # ← ユーザー入力
auto_runnerのディレクトリに移動したのを確認
(myvenv)
C:\Users\seraku\Documents\newtonx_project\newtonx_adk\adk_examples\auto_runner>
auto_runnerを動作させるのに必要な環境をインストール
(myvenv) pip install -r requirements.txt # ← ユーザー入力
```

### 参考：pip install -r requirements.txtについて

**目的：** requirements.txtというファイルに記述された依存関係を一括でインストールします。

**動作：** requirements.txtには、プロジェクトに必要なPythonパッケージとそのバージョンが一覧として記述されています。 pip (Pythonのパッケージ管理ツール) がこのファイルを読み取り、指定されたパッケージをインターネット上のPython Package Index (PyPI) からダウンロードしてインストールします。

#### 必要なものがあればどうするのか？

##### 1. 必要な依存関係が不足している場合

- **問題：** プロジェクトを動かそうとした際、必要なライブラリがインストールされていないとエラーが発生します。
- **解決方法：** 開発者が依存関係をrequirements.txtに追加する。追加後、再度以下のコマンドを実行する：

```
pip install -r requirements.txt
```

##### 2. 依存関係のバージョンが不適切な場合

- **問題：** バージョンの不一致により動作が不安定になる場合があります。
- **解決方法：** requirements.txtの中で特定のバージョンを指定します (例: numpy==1.21.0)。バージョン指定後に再インストールします。

## 6.2 サンプルのターミナルからの実行方法

1. cli\_example.pyを実行

```
必ずコマンドプロンプトの先頭に仮想環境名(myvenv)が表示されている状態で実施してください。
newtonx_projectのディレクトリに一旦もどりましょう。今いる場所で以下を入力してください。
(myvenv) ...> cd C:\Users\seraku\Documents\newtonx_project # ← ユーザー入力
(myvenv) C:\Users\seraku\Documents\newtonx_project> cd
newtonx_adk\adk_examples\cli_example # ← ユーザー入力
(myvenv)
C:\Users\seraku\Documents\newtonx_project\newtonx_adk\adk_examples\cli_example> python cli_example.py # ← ユーザー入力
```

macOSの場合:

```
cd newtonx_adk/adk_examples/cli_example
python3 cli_example.py
```

## 2. expense\_processor\_example.pyを実行

```
必ずコマンドプロンプトの先頭に仮想環境名(myvenv)が表示されている状態で実施してください。
newtonx_projectのディレクトリに一旦もどりましょう。今いる場所で以下を入力してください。
(myvenv) ...> cd C:\Users\seraku\Documents\newtonx_project # ← ユーザー入力
(myvenv) C:\Users\seraku\Documents\newtonx_project> cd
newtonx_adk\adk_examples\expense_processor # ← ユーザー入力
(myvenv)
C:\Users\seraku\Documents\newtonx_project\newtonx_adk\adk_examples\expense_processor> python expense_processor_example.py sample_data output.csv # ← ユーザー入力
```

第1引数: レシートのイメージを格納するフォルダ  
第2引数: 結果出力のcsvファイル名

macOSの場合:

```
cd newtonx_adk/adk_examples/expense_processor
python3 expense_processor_example.py sample_data output.csv
```

注意: スクリプト実行前に仮想環境が有効化されていることを確認してください。

## 3. auto\_runner.pyを実行



```
必ずコマンドプロンプトの先頭に仮想環境名(myvenv)が表示されている状態で実施してください。📖
newtonx_projectのディレクトリに一旦もどりましょう。今いる場所で以下を入力してください。
(myvenv) ...> cd C:\Users\seraku\Documents\newtonx_project # ← ユーザー入力📖
(myvenv) C:\Users\seraku\Documents\newtonx_project>cd
newtonx_adk\adk_examples\auto_runner # ← ユーザー入力📖
(myvenv)
C:\Users\seraku\Documents\newtonx_project\newtonx_adk\adk_examples\auto_runner>python auto_runner.py -c config.yml # ← ユーザー入力📖
```

第1引数: -c (オプション)  
第2引数: yamlファイル名

macOSの場合:

```
cd newtonx_adk/adk_examples/auto_runner
python3 auto_runner.py -c config.yml
```

注意: スクリプト実行前に仮想環境が有効化されていることを確認してください。📖

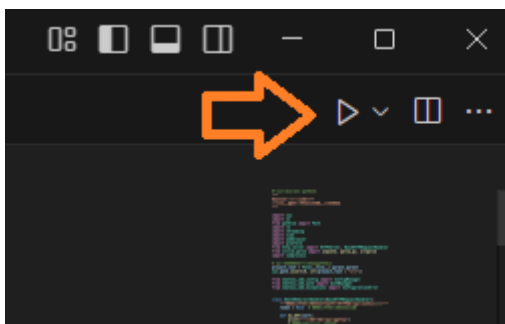
## 6.3 VSCodeから実行またはデバッグ実行 📖

### 📖 重要 📖

引数をもつアプリとそうでないものを切り替える場合には上記の操作を必ずしてください。その後は、**実行したい.pyファイルを表示してF5(デバッグ)またはCtrl+F5(高速実行)を押すと、直前で選んだ上記の環境を継続してアプリが実行されます。**

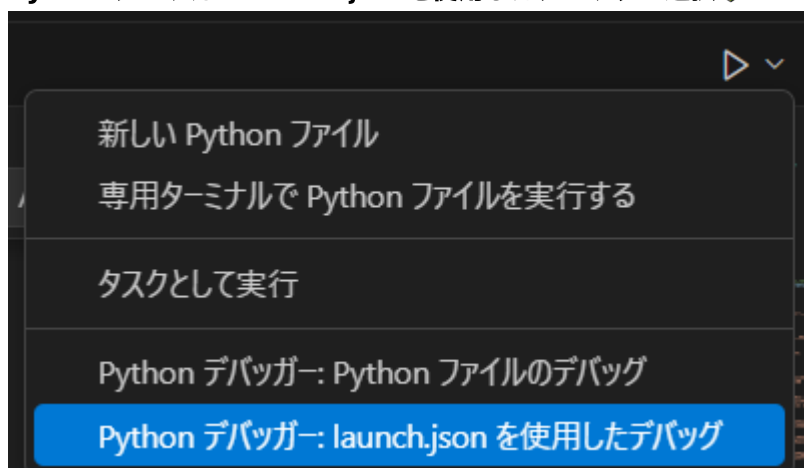
### 6.3.1 cli\_example.py (実行時に引数なしのアプリはこの方法で実行します) 📖

- **cli\_example.py** ファイルをエディタで開いて、Tabがアクティブな状態にします。(他のコードが前面にきていないこと)



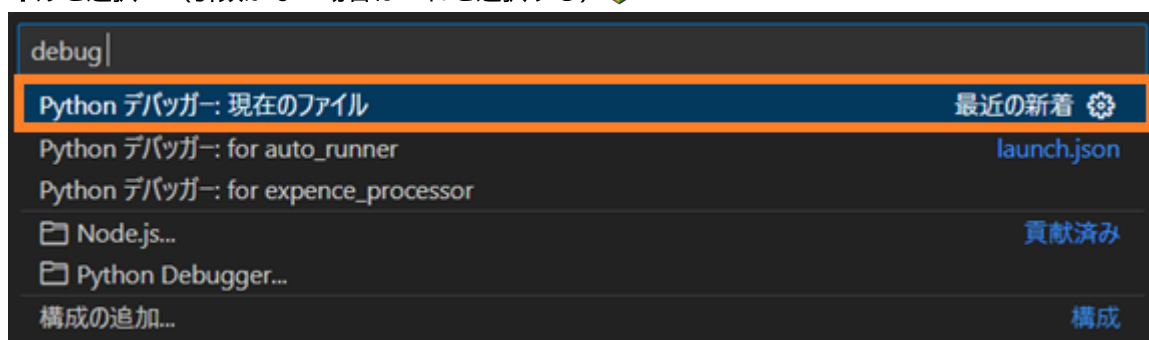
- 右上の「▶」ボタンをクリックします。📖

- Python デバッガー: launch.jsonを使用したデバッグを選択📖



- Python デバッガー:現在のファ

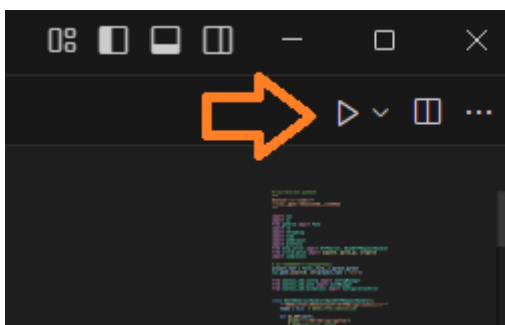
イルを選択 （引数がない場合はこれを選択する）📖



- 一度デバッグ起動されますが、その後、F5キー（デバッグ実行）、Ctrl+F5（高速実行）で、同様の環境で継続してワンタッチで起動が可能です。
- （macOS）Ctrl + F5 で同様に実行できます

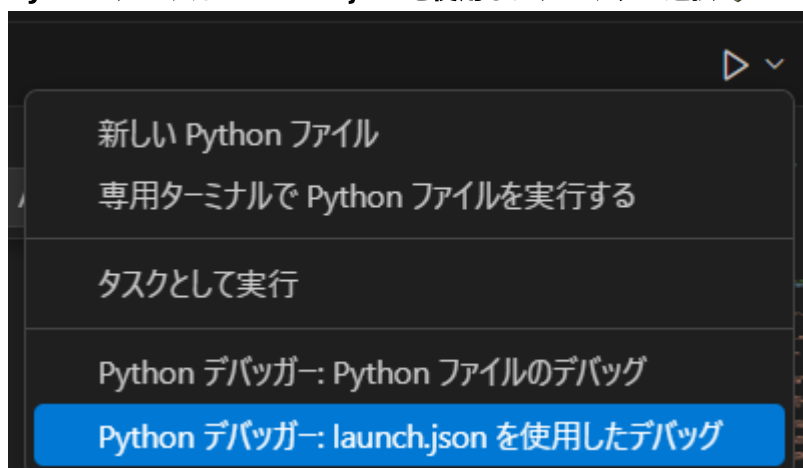
### 6.3.2 expense\_processor\_example.py （実行時に引数あり）📖

- expense\_processor\_example.pyファイルをエディタで開いて、コードを表示するTabがアクティブな状態にします。（他のコードが前面にきていないこと）

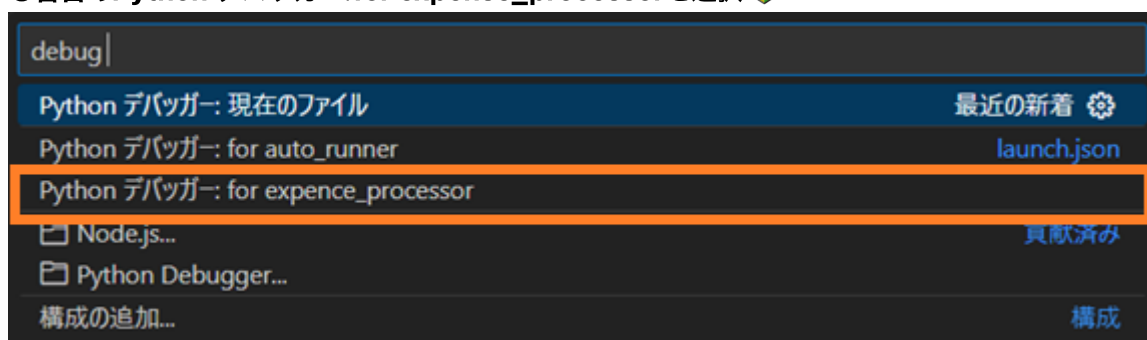


- 右上の「▶」ボタンをクリックします。📖

- Python デバッガー: launch.jsonを使用したデバッグを選択 📖



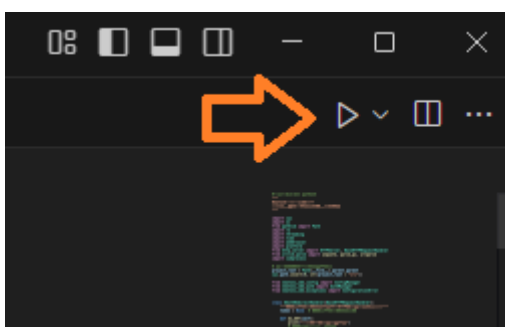
- 3番目のPython デバッガー: for expense\_processorを選択 📖



- 一度デバッグ起動されますが、その後、F5キー（デバッグ実行）、Ctrl+F5（高速実行）で、同様の環境で継続してワンタッチで起動が可能です。
- （macOS）F5（デバッグ実行）、Ctrl + F5（デバッグなし実行）で同様に実行できます

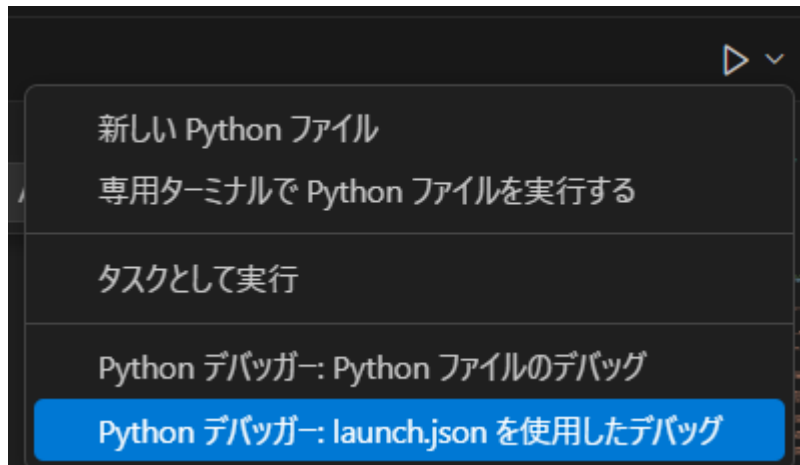
### 6.3.3 auto\_runner.py （実行時に引数あり） 📖

- auto\_runner.pyファイルをエディタで開いて、コードを表示するTabがアクティブな状態にします。（他のコードが前面にきていないこと）

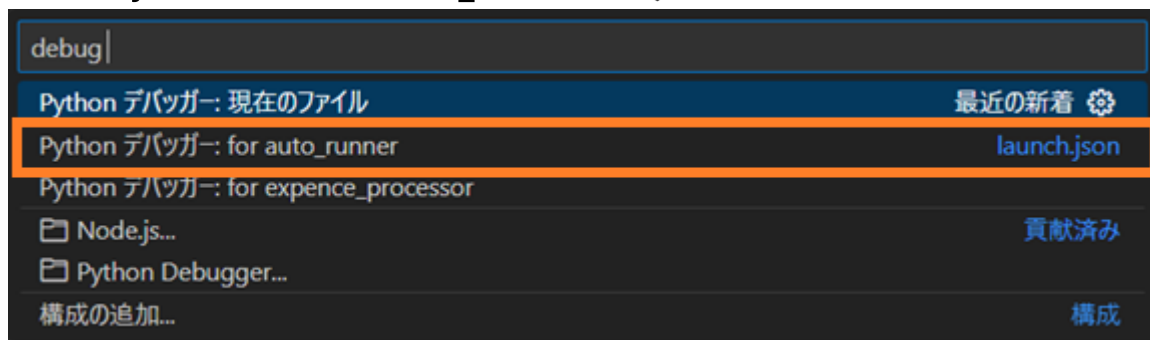


- 右上の「▶」ボタンをクリックします。 📖

- Python デバッガー: launch.jsonを使用したデバッグを選択



- 2 番目のPython デバッガー: for auto\_runnerを選択



- 一度デバッグ起動されますが、その後、F5キー（デバッグ実行）、Ctrl+F5（高速実行）で、同様の環境で継続してワンタッチで起動が可能です。
- （macOS）F5（デバッグ実行）、Ctrl + F5（デバッグなし実行）で同様に実行できます

## 7. サンプルアプリケーションの紹介

NewtonX ADKには、3つのサンプルアプリケーションが含まれています。各サンプルの実行方法を、VSCodeからの実行とターミナルからの実行の両方で説明します。

### adk\_examples ディレクトリ構成

```
newtonx_adk/
└─ adk_examples/
 ├── cli_example/ # サンプル1
 ├── expense_processor/ # サンプル2
 └─ auto_runner/ # サンプル3
```

#### 7.1 サンプル説明

##### 7.1.1 サンプル1: CLI Example

NewtonX ADKを使用したCLIアプリケーションの実装例です。**サンプルで実行するためのフォルダ、および、画像ファイルはADKに同梱されています。**

#### 実行できる機能一覧

今回、ADKで提供される個別の機能をメニューから選択実行できます。アプリ作成時に各個別機能呼び出す方法の実装例として参考にしてください。

- **タブキー補完:** ファイルパス入力時にタブキーで自動補完
- **ファイルアップロード:** 画像・ドキュメントファイルのアップロード
- **チャット機能:** チャット作成、リアルタイムチャット実行
- **アシスタント管理:** アシスタント一覧表示・選択・ナレッジ登録・ナレッジ削除
- **履歴機能:** コマンド履歴の保存・参照

##### 7.1.2 サンプル2: Expense Processor（経費処理）

このプログラムは、引数で指定したフォルダ下に配置したレシートの画像を、AIで分析し、経費精算に必要な項目を抽出し、引数で指定したファイルにCSV出力します。

##### 7.1.3 サンプル3: Auto Runner（自動実行）

このサンプルは設定ファイル（config.yml）に指示を記載し自動実行するアプリケーションです。実行される内容と、以下のymlファイルに記載されている内容を比較してください。このファイルを書き換えることにより、自動実行で行われる処理を変更することができます。

#### 機能

- **設定ファイルベース:** YAML設定ファイルによる自動実行
- **タスク定義:** 複数のタスクを順次実行
- **進捗表示:** Richライブラリによる美しい進捗表示

- **結果保存:** 実行結果の自動保存
- **ログ機能:** 詳細なログ出力

### 7.1.3.1 Auto Runnerで使用する設定ファイルの準備

#### 1. config.ymlファイルの確認（ADKに同梱されていますので通常は新規作成不要です。）

- `newtonx_adk/adk_examples/auto_runner`ディレクトリに同梱されている`config.yml`ファイルが存在することを確認します。
- 必要に応じて内容を編集します。以下はサンプルとなる内容です：

```
NewtonX Auto Runner Configuration
YAML設定ファイルに従って自動実行するサンプル

project:
 name: "NewtonX Auto Runner Example"
 description: "設定ファイルベースの自動実行サンプル"
 version: "1.0.0"

認証設定
auth:
 auto_login: true
 session_timeout: 3600

チャット設定
chat:
 default_assistant: "高速アシスタント"
 search_types:
 - "WEB検索（最新情報）"
 - "ナレッジ検索（登録ファイル）"
 - "両方（制限あり）"
 - "検索なし（基本対話）"

ファイルアップロード設定
upload:
 max_file_size: 10485760 # 10MB
 allowed_extensions:
 images: [".jpg", ".jpeg", ".png", ".gif", ".bmp", ".webp"]
 documents: [".pdf", ".docx", ".txt", ".pptx", ".xls", ".xlsx", ".md"]

自動実行タスク
tasks:
 - name: "初期化"
 type: "init"
 description: "プロジェクトの初期化"
 enabled: true

 - name: "チャット作成"
 type: "create_chat"
 description: "新しいチャットの作成"
 title: "Auto Runner Test Chat"
 assistant: "高速アシスタント"
 enabled: true
```

```
- name: "ファイルアップロード"
 type: "upload"
 description: "サンプルファイルのアップロード"
 files:
 - "sample_document.txt"
 enabled: true

- name: "ファイル分析"
 type: "analyze_file"
 description: "アップロードしたファイルの分析"
 message: "このファイルの中身について教えてください。"
 enabled: true

- name: "メッセージ送信"
 type: "send_message"
 description: "テストメッセージの送信"
 message: "こんにちは！Auto Runnerのテストです。"
 web_search: false
 knowledge_search: false
 enabled: true

ログ設定
logging:
 level: "INFO"
 file: "auto_runner.log"
 max_size: 10485760 # 10MB
 backup_count: 5

出力設定
output:
 format: "rich" # rich, json, plain
 save_results: true
 output_dir: "results"
```

### 7.1.3.2 設定ファイル (config.yml) の説明

Auto Runnerサンプルで使用する`config.yml`ファイルの各セクションについて説明します：

#### プロジェクト情報 (project)

- `name`: プロジェクト名
- `description`: プロジェクトの説明
- `version`: バージョン情報

#### 認証設定 (auth)

- `auto_login`: 自動ログインの有効/無効
- `session_timeout`: セッションタイムアウト時間 (秒)

#### チャット設定 (chat)

- **default\_assistant**: デフォルトのアシスタント
- **search\_types**: 利用可能な検索タイプの一覧

### ファイルアップロード設定 (upload)

- **max\_file\_size**: 最大ファイルサイズ (バイト)
- **allowed\_extensions**: 許可されるファイル拡張子の分類

### 自動実行タスク (tasks)

各タスクの設定項目：

- **name**: タスク名
- **type**: タスクタイプ (init, create\_chat, upload, analyze\_file, send\_message)
- **description**: タスクの説明
- **enabled**: タスクの有効/無効

### ログ設定 (logging)

- **level**: ログレベル (INFO, DEBUG, WARNING, ERROR)
- **file**: ログファイル名
- **max\_size**: ログファイルの最大サイズ
- **backup\_count**: ロテーション時のバックアップ数

### 出力設定 (output)

- **format**: 出力形式 (rich, json, plain)
- **save\_results**: 結果の保存有無
- **output\_dir**: 出力ディレクトリ

参考 tasks.json

## 1. tasks.json の役割と動作

### 役割

- **ビルドやテスト、自動化処理の定義**を行うための設定ファイルです。
- Pythonでは、例えば以下のようなタスクを定義できます：
  - コードの静的解析 (例: **pylint**、**flake8**)
  - テストの実行 (例: **pytest**、**unittest**)
  - 環境セットアップ (例: 仮想環境の作成や依存パッケージのインストール)
  - コンパイルやスクリプトの実行

### 参照されるタイミング

1. 「タスクの実行」メニューから明示的に選択したとき
  - **Ctrl + Shift + P** → 「Run Task」を選択し、リストからタスクを選んで実行したときに参照されます。
  - これにより、**tasks.json** に定義されたタスクを手動で実行できます。



- **label**: タスク名。Ctrl + Shift + P → 「Run Task」でこの名前が表示されます。
- **group**: タスクの種類を指定できます (build, test など)。isDefault を true にすると、ショートカットキー (例: Ctrl + Shift + B) で直接実行できます。

## 2. 本ガイドでのtasks.json設定例

1. Windows用 ※仮想環境名がmyvenvでない場合は、該当部分を変更してください。

```
{
 "version": "2.0.0",
 "tasks": [
 {
 "label": "Run current file with venv Python",
 "type": "shell",
 "command": "${workspaceFolder}/myvenv/Scripts/python.exe",
 "args": ["${file}"],
 "options": {
 "cwd": "${fileDirname}",
 "env": {
 "PYTHONPATH": "${workspaceFolder}/myvenv/Lib/site-
packages"
 }
 },
 "group": {
 "kind": "build",
 "isDefault": false
 },
 "presentation": {
 "echo": true,
 "reveal": "always",
 "focus": false,
 "panel": "shared"
 },
 "problemMatcher": []
 },
 {
 "label": "Run auto_runner with config",
 "type": "shell",
 "command": "python",
 "args": ["${file}", "-c", "${fileDirname}/config.yml"], ←引数指
 "group": {
 "kind": "build",
 "isDefault": true
 },
 "presentation": {
 "echo": true,
 "reveal": "always",
 "focus": false,
 "panel": "shared"
 }
 }
]
}
```

```

 "problemMatcher": []
 },
 {
 "label": "Run expense_processor with sample_data and
output.csv",
 "type": "shell",
 "command": "python",
 "args": ["${file}", "${fileDirname}/sample_data",
"${fileDirname}/output.csv"], ←引数指定
 "group": {
 "kind": "build",
 "isDefault": false
 },
 "presentation": {
 "echo": true,
 "reveal": "always",
 "focus": false,
 "panel": "shared"
 },
 "problemMatcher": []
 }
]
}

```

2. Mac用 ※仮想環境名が**myvenv**でない場合は、該当部分を変更してください。

```

{
 "version": "2.0.0",
 "tasks": [
 {
 "label": "Run current file (no args) - mac",
 "type": "shell",
 "command": "${workspaceFolder}/myvenv/bin/python",
 "args": [
 "${file}"
],
 "group": {
 "kind": "build",
 "isDefault": true
 },
 "options": {
 "cwd": "${fileDirname}"
 },
 "presentation": {
 "echo": true,
 "reveal": "always",
 "focus": false,
 "panel": "shared"
 },
 "problemMatcher": []
 },
]
}

```

```

 "label": "Run auto_runner with config",
 "type": "shell",
 "command": "${workspaceFolder}/myvenv/bin/python",
 "args": [
 "${file}",
 "-c",
 "${fileDirname}/config.yml"
], ←引数を指示
 "group": {
 "kind": "build",
 "isDefault": true
 },
 "options": {
 "cwd": "${fileDirname}"
 },
 "presentation": {
 "echo": true,
 "reveal": "always",
 "focus": false,
 "panel": "shared"
 },
 "problemMatcher": []
 },
 {
 "label": "Run expense_processor with sample_data and
output.csv",
 "type": "shell",
 "command": "${workspaceFolder}/myvenv/bin/python",
 "args": [
 "${file}",
 "${fileDirname}/sample_data",
 "${fileDirname}/output.csv"
], ←引数を指示
 "group": {
 "kind": "build",
 "isDefault": false
 },
 "options": {
 "cwd": "${fileDirname}"
 },
 "presentation": {
 "echo": true,
 "reveal": "always",
 "focus": false,
 "panel": "shared"
 },
 "problemMatcher": []
 }
]
}

```

参考 launch.json

1. launch.json の役割と動作

役割

- プログラムを **デバッグ実行** するための設定を行うファイルです。
- デバッグ対象のスクリプト、引数、環境変数、ブレークポイントやリモートデバッグの設定などを指定します。
- Pythonのデバッグを含むさまざまなプログラムのデバッグ設定が可能です。

参照されるタイミング

1. デバッグを開始したとき
  - **F5**キー、または **Ctrl + Shift + D** → **デバッグ構成を選択して再生ボタンをクリック** した際に参照されます。
- **name**: デバッグ構成名。デバッグ開始時に選択されるリストに表示されます。
  - **type**: デバッグ対象の言語や環境（Pythonの場合は **"python"**）。
  - **request**: デバッグの種類（**launch** はローカル実行、**attach** はリモートデバッグ）。
  - **program**: 実行するスクリプトのパス。
  - **preLaunchTask**: デバッグ開始前に実行する **tasks.json** のタスク名。

2. まとめ

項目	tasks.json	launch.json
役割	ビルド、テスト、静的解析などのタスクを定義し、自動化するための設定ファイル。	プログラムのデバッグ設定を行うためのファイル。
参照されるタイミング	1. <b>Ctrl + Shift + P</b> → 「Run Task」でタスクを実行する際。 2. <b>launch.json</b> の <b>preLaunchTask</b> 設定が指定されている場合、デバッグ開始前に実行される。	1. F5キーやデバッグ構成の選択時（デバッグ開始時）。 2. デバッグ開始時に <b>preLaunchTask</b> を参照して、 <b>tasks.json</b> のタスクを実行する場合がある。
例	静的解析（pylint）、テスト実行、依存関係のインストールなどのタスクを定義。	実行ファイル、引数、デバッグ設定（ブレークポイント、環境変数、リモートデバッグ）を指定。

3. 本ガイドでのlaunch.json設定例

1. Windows用 ※仮想環境名がmyvenvでない場合は、該当部分を変更してください。

```
{
 "version": "0.2.0",
 "configurations": [
 {
```

```

 "name": "Python デバッガー: 現在のファイル",
 "type": "debugpy",
 "request": "launch",
 "program": "${file}",
 "console": "integratedTerminal",
 "justMyCode": false,
 "cwd": "${fileDirname}",
 "env": {
 "PYTHONPATH": "${workspaceFolder}\\myvenv\\Lib\\site-
packages"
 },
 },
 {
 "name": "Python デバッガー: for auto_runner",
 "type": "debugpy",
 "request": "launch",
 "program": "${file}",
 "console": "integratedTerminal",
 "justMyCode": false,
 "env": {
 "PYTHONPATH": "${workspaceFolder}\\myvenv\\Lib\\site-
packages"
 },
 "cwd": "${fileDirname}",
 "args": ["-c", "config.yml"]
 },
 {
 "name": "Python デバッガー: for expense_processor",
 "type": "debugpy",
 "request": "launch",
 "program": "${file}",
 "console": "integratedTerminal",
 "justMyCode": false,
 "env": {
 "PYTHONPATH": "${workspaceFolder}\\myvenv\\Lib\\site-
packages"
 },
 "cwd": "${fileDirname}",
 "args": ["sample_data", "output.csv"]
 }
]
}

```

2. Mac用 ※仮想環境名がmyvenvでない場合は、該当部分を変更してください。

```

{
 "version": "0.2.0",
 "configurations": [
 {
 "name": "Python デバッガー: 現在のファイル",
 "type": "debugpy",
 "request": "launch",

```

```

 "program": "${file}",
 "console": "integratedTerminal",
 "justMyCode": false,
 "cwd": "${fileDirname}",
 "env": {
 "PYTHONPATH":
"${workspaceFolder}/myenv/lib/python3.11/site-packages"
 },
 "python": "${workspaceFolder}/myenv/bin/python"
 },
 {
 "name": "Python デバッガー: for auto_runner",
 "type": "debugpy",
 "request": "launch",
 "program": "${file}",
 "console": "integratedTerminal",
 "justMyCode": false,
 "cwd": "${fileDirname}",
 "args": ["-c", "config.yml"],
 "env": {
 "PYTHONPATH":
"${workspaceFolder}/myenv/lib/python3.11/site-packages"
 },
 "python": "${workspaceFolder}/myenv/bin/python"
 },
 {
 "name": "Python デバッガー: for expense_processor",
 "type": "debugpy",
 "request": "launch",
 "program": "${file}",
 "console": "integratedTerminal",
 "justMyCode": false,
 "cwd": "${fileDirname}",
 "args": ["sample_data", "output.csv"],
 "env": {
 "PYTHONPATH":
"${workspaceFolder}/myenv/lib/python3.11/site-packages"
 },
 "python": "${workspaceFolder}/myenv/bin/python"
 }
]
}

```

## 8. macOSクイックリファレンス（コマンドとショートカット）

- ターミナル起動: Spotlightで「Terminal」、またはApplications → Utilities → Terminal
- venv有効化/無効化:

```
source myvenv/bin/activate
deactivate
```

- Python/Pipコマンド: `python3`, `pip3` を推奨
- パス確認: `which python3`, `which pip3`
- VSCodeショートカット:
  - コマンドパレット: Cmd + Shift + P
  - 設定: Cmd + ,
  - 統合ターミナル表示/非表示: Ctrl + `
  - デバッグ実行: F5
  - デバッグなし実行: Ctrl + F5
- VSCode 統合ターミナルの既定シェル: 「Terminal: Select Default Profile」 → zsh（推奨）
- NewtonX設定ファイル: `/Users/<ユーザー名>/.newtonx/config.json`